

Practical Workbook

<Software Design & Architecture>

<(SE-308)>



Name: _____

Seat #: _____

Batch: _____

Semester: _____

Year: _____

Practical Workbook

< Software Design & Architecture >

<(SE-308)>



Prepared By

<Ms. Sana Fatima>
<Lecturer>

Approved By

Prof. Dr. Shehnida Zardari
Chairperson

DEPARTMENT OF SOFTWARE ENGINEERING, NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Contents

Lab #	Lab Objects	Page #	Rubrics	Signature
1.	Introduction to Unified Modeling Language (UML)	4		
2.	Install & identify components of StarUML	10	✓	
	Identify and describe the main elements of a Use Case diagram	14		
3.	Interpret and model real-world problems into Class Diagrams	20	✓	
4.	Model real-world scenarios into State Transition Diagrams	24		
5.	Model real-world business processes using Activity Diagram	27	✓	
6.	Model real-world scenarios into Sequence Diagrams	32		
7.	Open Ended Lb Error! Bookmark not defined.		OEL	
8.	35			
9.	Model real-world scenarios into Dataflow Diagram	38		
10.	Model real-world Systems into Deployment Diagram	42		
11.	Model real-world Systems into communication Diagram	45		
12.	Getting Familiar with FIGMA	48	CEA	
13.	Complex Engineering Activity	52		
14.	To Learn UI / UX Design with Wireframe in Figma	55	FINAL	
15.	Final	60		

LAB SESSION 01

Introduction to Unified Modeling Language (UML)

THEORY

The Unified Modeling Language (UML) enables system builders to create blueprints that capture their visions in a standard, easy-to understand way, and provides a mechanism to effectively share and communicate these visions with others. It does this via a set of symbols and diagrams. Each diagram plays a different role within the development process.

COMPONENTS OF UML

- The UML consists of a number of graphical elements that combine to form diagrams.
- UML is a language; it has rules for combining these elements.
- The purpose of the diagrams is to present multiple views of a system; this set of multiple views is called a model.
- A UML model tells what a system is supposed to do.

PURPOSE OF UML

"An image speaks louder than words," and this is perfectly applicable when discussing UML. Object-oriented concepts were introduced before UML, and at that time, there were no standardized methods for organizing and consolidating object-oriented development. This is when UML was introduced. While there are many goals for creating UML, the key objective is:

To establish a general-purpose modeling language that all modelers can use, and it should also be simple to understand and apply. It is designed for developers, but also for business users, common people, and anyone wanting to understand the system. The system could be either software or non-software. It is important to note that UML is not a development method, but it supports processes to ensure the success of a system. In conclusion, the purpose of UML can be defined as providing a simple modeling tool for modeling any practical systems in today's complex environment.

GOALS OF UML

A picture is worth a thousand words, this idiom absolutely fits describing UML. Object-oriented concepts were introduced much earlier than UML. At that point of time, there were no standard methodologies to organize and consolidate the object-oriented development. It was then that UML came into picture.

There are a number of goals for developing UML but the most important is to define some general purpose modeling language, which all modelers can use and it also needs to be made simple to understand and use.

UML diagrams are not only made for developers but also for business users, common people, and anybody interested to understand the system. The system can be a software or non-software system. Thus it must be clear that UML is not a development method rather it accompanies with processes to make it a successful system.

In conclusion, the goal of UML can be defined as a simple modeling mechanism to model all possible practical systems in today's complex environment.

A CONCEPTUAL MODEL OF UML

To understand the conceptual model of UML, first we need to clarify what is a conceptual model? and why a conceptual model is required?

- A conceptual model can be defined as a model which is made of concepts and their relationships.
- A conceptual model is the first step before drawing a UML diagram. It helps to understand the entities in the real world and how they interact with each other.

As UML describes the real-time systems, it is very important to make a conceptual model and then proceed gradually. The conceptual model of UML can be mastered by learning the following three major elements –

- UML building blocks
- Rules to connect the building blocks
- Common mechanisms of UML

UML - Building Blocks

As UML describes the real-time systems, it is very important to make a conceptual model and then proceed gradually. The conceptual model of UML can be mastered by learning the following three major elements –

- UML building blocks
- Rules to connect the building blocks
- Common mechanisms of UML

The building blocks of UML can be defined as –

1. Things
2. Relationships
3. Diagrams

1. **Things** are the most important building blocks of UML. Things can be –

- a. Structural
- b. Behavioral
- c. Grouping
- d. Annotational

a. **Structural Things** Structural things define the static part of the model. They represent the physical and conceptual elements. Following are the brief descriptions of the structural things. Such as Class, Interface, Collaboration, Usecase, Component and Nodes

✓ **Class** – Class represents a set of objects having similar responsibilities.

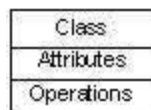


Fig 1.1 Class Representation in UML

✓ **Interface** – Interface defines a set of operations, which specify the responsibility of a class.

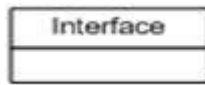


Fig 1.2 Interface Representation

- ✓ **Collaboration** –Collaboration defines an interaction between elements.



Fig 1.3 Collaboration Representation

- ✓ **Use case** –Use case represents a set of actions performed by a system for a specific goal.

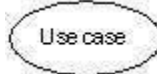


Fig 1.4 Usecase Representation

- ✓ **Component** –Component describes the physical part of a system.

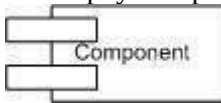


Fig 1.5 Component Representation

- ✓ **Node** – A node can be defined as a physical element that exists at run time.

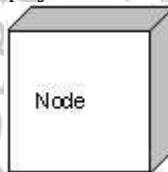


Fig. 1.6 Node representation

- b. **Behavioral Things:** A behavioral thing consists of the dynamic parts of UML models. Following are the behavioral things –

- ✓ **Interaction** – Interaction is defined as a behavior that consists of a group of messages exchanged among elements to accomplish a specific task.

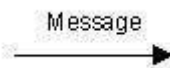


Fig. 1.7 Message representation

- ✓ **State machine** – State machine is useful when the state of an object in its life cycle is important. It defines the sequence of states an object goes through in response to events. Events are external factors responsible for state change

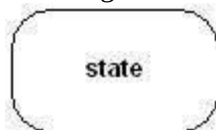


Fig. 1.8 State representation

- c. **Grouping Things:** Grouping things can be defined as a mechanism to group elements of a UML model together. There is only one grouping thing available –

- ✓ **Package** – Package is the only one grouping thing available for gathering structural and behavioral things.



Fig. 1.9 State representation

- d. **Annotational Things:** Annotational things can be defined as a mechanism to capture remarks, descriptions, and comments of UML model elements. **Note** - It is the only one Annotational thing available. A note is used to render comments, constraints, etc. of an UML element.

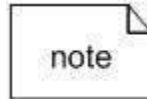


Fig. 1.10 Note representation

2. Relationship: is another most important building block of UML. It shows how the elements are associated with each other and this association describes the functionality of an application. There are four kinds of relationships available.

a. Dependency

Dependency is a relationship between two things in which change in one element also affects the other.

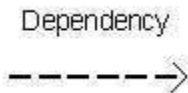


Fig. 1.11 Dependency representation

b. Association

Association is basically a set of links that connects the elements of a UML model. It also describes how many objects are taking part in that relationship.



Fig. 1.12 Association representation

c. Generalization

Generalization can be defined as a relationship which connects a specialized element with a generalized element. It basically describes the inheritance relationship in the world of objects.

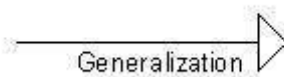


Fig. 1.13 Generalization representation

d. Realization

Realization can be defined as a relationship in which two elements are connected. One element describes some responsibility, which is not implemented and the other one implements them. This relationship exists in case of interfaces.

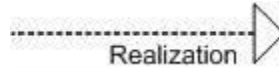


Fig. 1.14 Realization representation

3. **UML Diagrams:** UML diagrams are the ultimate output of the entire discussion. All the elements, relationships are used to make a complete UML diagram and the diagram represents a system. The visual effect of the UML diagram is the most important part of the entire process. All the other elements are used to make it complete.

CHALLENGES OF UML IN SOFTWARE DEVELOPMENT

- Diagrams Versus Implementation
- Ambiguity and Interpretation
- Overwhelming Complexity
- Limited Tool Integration
- Maintenance Challenges
- Communication Issues

4+1 VIEW MODEL

The 4+1 View Model was designed by Philippe Kruchten to describe the architecture of a software system based on the use of multiple and concurrent views. It is a multiple view model that addresses different features and concerns of the system. It standardizes the software design documents and makes the design easy to understand by all stakeholders.

It is an architecture verification method for studying and documenting software architecture design and covers all the aspects of software architecture for all stakeholders. It provides four essential views –

- The logical view or conceptual view – It describes the object model of the design.
- The process view – It describes the activities of the system, captures the concurrency and synchronization aspects of the design.
- The physical view – It describes the mapping of software onto hardware and reflects its distributed aspect.
- The development view – It describes the static organization or structure of the software in its development of environment.

This view model can be extended by adding one more view called scenario view or use case view for end-users or customers of software systems. It is coherent with other four views and are utilized to illustrate the architecture serving as plus one view, (4+1) view model.

EXERCISE

1. List the advantages of Unified Modeling Language.

2. Explain different kinds of UML diagram and their significance

3. Explain Some possible challenges of UML in software development in your own words

4. State some real world Applications of UML

5. State 4+1 view Model and why it is called 4+1 instead of 5 View model

LAB SESSION 02

Install & Identify key features and interface components of StarUML

THEORY

StarUML is an open source software modeling tool that supports the UML (Unified Modeling Language) framework for system and software modeling. It actively supports the MDA (Model Driven Architecture) approach by supporting the UML profile concept and allowing to generate code for multiple languages. The key features of StarUML are:

- Multi-platform support (Mac OS, Windows and Linux)
- UML 2.x standard compliant
- SysML support
- Entity-Relationship diagram (ERD)
- Data-flow diagram (DFD)
- Flowchart diagram
- Multiple windows
- Modern UXDark and light themes
- Retina (High-DPI) display support
- MacPro Pro's Touch Bar support
- Model-driven development
- Open APIs
- Various third-party extensions
- Asynchronous model validation
- Export to HTML docs
- Automatic updates.

Main Window

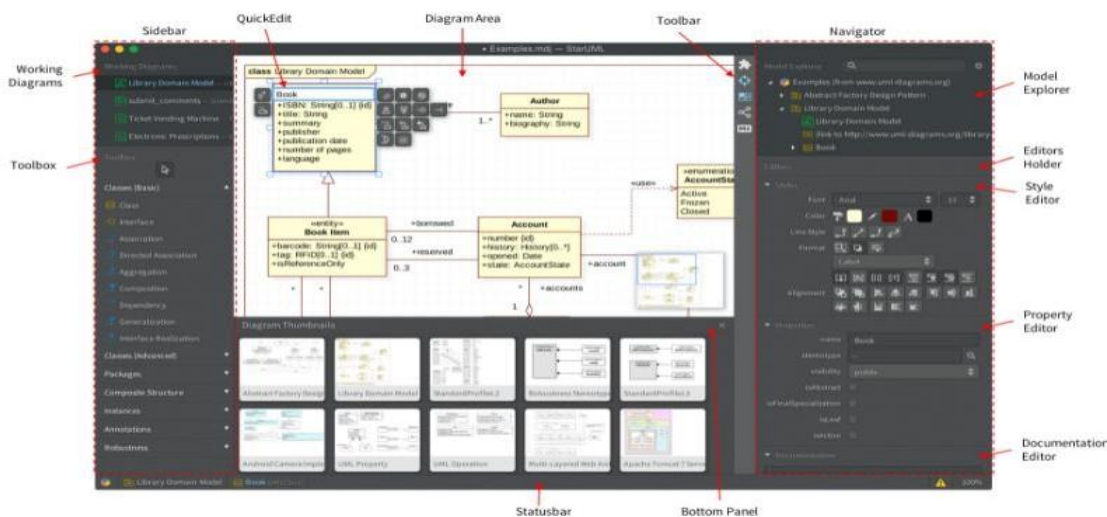


Fig. 2.1 staruml Main Window

- ✓ **Diagram Area** *Diagram Area* shows the currently selected diagram.
- ✓ **Sidebar** *Side bar* is the area left containing **Working Diagrams** panel and **Toolbox**. To show or hide **Sidebar**, press **Ctrl+1** or check (or uncheck) **View | Sidebar** in Menu Bar.
- ✓ **Working Diagrams** *Working Diagrams* panel shows a list of the opened working diagram. The selected diagram is shown in the **Diagram Area**.
- ✓ **Toolbox** *Tool box* shows elements which can be created in the selected diagram. To create an element: 1. Select an element in **Toolbox** 2. Mouse down on the **Diagram Area** and then drag as the size of the element to be created. If the element is a kind of relationship, connect two elements by drag and drop. To show or hide **Toolbox**, press **Ctrl+5** or check (or uncheck) **View | Toolbox** in Menu Bar.
- ✓ **Editors (Holder)** *Editors (Holder)* contains editors to edit properties of model and view elements. It includes **Style Editor**, **Property Editor**, and **Documentation Editor**. To show or hide **Editors**, press **Ctrl+6** or check (or uncheck) **View | Editors** in Menu Bar.
- ✓ **Style Editor** *Style Editor* allows to edit styles of selected view elements.
- ✓ **Property Editor** *Property Editor* allows to edit properties of selected model elements.
- ✓ **Documentation Editor** *Documentation Editor* allows to edit documentation property of a selected model element.
- ✓ **Navigator** is the right area containing **Model Explorer** and **Editors**. To show or hide **Navigator**, press **Ctrl+2** or check (or uncheck) **View | Navigator** in Menu Bar.
- ✓ **Model Explorer** *Model Explorer* shows the tree structure of model elements. You can find an element quickly by search box. To change the order of elements: 1. Select the **Model Explorer Settings** menu in the right up corner of **Model Explorer**. 2. Select one of the sorting types
- ✓ To show or hide stereotypes in front of element name by check of uncheck **Show Stereotype Text** in the **Model Explorer Settings** menu.
- ✓ **Toolbar** *Toolbar* shows tool buttons typically provided from default or installed third-party extensions. To show or hide **Toolbar**, press **Ctrl+3** or check (or uncheck) **View | Toolbar** in Menu Bar.
- ✓ **Bottom Panel** *Bottom Panel* is a panel shown below **Diagram Area** typically provided from default or installed third-party extensions including **Find Results**, **Diagram Thumbnails**, **Validation Results**, **Markdown Editor** and etc.

- ✓ **Statusbar** To show or hide **Statusbar**, press **Ctrl+4** or check (or uncheck) View | Statusbar in Menu Bar.

EXERCISE

1. Install Staruml and state its key features in your own words

3. Create a dummy project and insert any UML diagram and set properties through Editor Holders and save it.

4. Compare StarUML with any other similar tool and state their significance and limitations.

NED University of Engineering & Technology
Department of Software Engineering
SE-308 Software Design & Architecture



Criteria	(5)	(4)	(3)	(2)	(1)	(0)
Problem Understanding & Objective Clarity	Demonstrates thorough understanding of lab objectives; clearly explains purpose and relevance.	Good understanding; objectives mostly clear with minor gaps.	Adequate understanding; some confusion in purpose.	Partial understanding; objectives vague.	Very little understanding; objectives unclear.	Did not attempt or irrelevant.
Experimental Design & Procedure	Procedure is logical, detailed, and accurate; anticipates possible errors.	Procedure is clear and mostly accurate; minor improvements needed.	Procedure covers basic steps but lacks depth or detail.	Incomplete procedure; several errors.	Poorly structured procedure; many inaccuracies.	No procedure attempted.
Data Collection & Accuracy	Data collected systematically, highly accurate, and well-documented.	Data mostly accurate and properly recorded; minor errors.	Data recorded but with some inconsistencies.	Data incomplete or error-prone.	Very limited data; mostly incorrect.	No data collected.
Analysis & Interpretation	Analysis is thorough, logical, and demonstrates deep understanding; conclusions fully supported.	Analysis is good with minor errors; conclusions mostly supported.	Adequate analysis; some conclusions not fully justified.	Limited analysis; weak interpretation.	Very poor analysis; conclusions invalid.	No analysis attempted.
Reporting, Presentation & Professionalism	Report/presentation is clear, well-organized, professional, with excellent use of visuals/tables.	Well-organized with minor issues in clarity or format.	Report/presentation understandable but lacks polish.	Disorganized; difficult to follow.	Very poorly written/presented; unclear.	No report/presentation submitted.
Weighted CLO Score						
Remarks and Signature						

LAB SESSION 03

Identify and describe the main elements of a Use Case diagram (actors, use cases, relationships)

THEORY

Use case diagrams describe what a system does from the standpoint of an external observer. The emphasis is on what a system does rather than how. Use case diagrams are closely connected to scenarios. A scenario is an example of what happens when someone interacts with the system. Here is a scenario for a medical clinic.

"A patient calls the clinic to make an appointment for a yearly checkup. The receptionist finds the nearest empty time slot in the appointment book and schedules the appointment for that time slot." A use case is a summary of scenarios for a single task or goal.

An actor is who or what initiates the events involved in that task. Actors are simply roles that people or objects play. The picture below is a Make Appointment use case for the medical clinic. The actor is a Patient. The connection between actor and use case is a communication association (or communication for short).

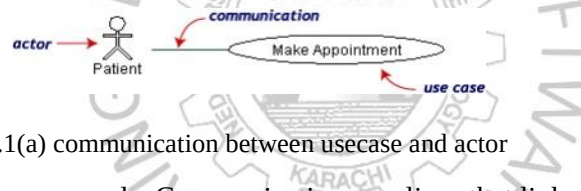


Fig 3.1(a) communication between usecase and actor

Actors are stick figures. Use cases are ovals. Communications are lines that link actors to use cases. A use case diagram is a collection of actors, use cases, and their communications. We've put Make Appointment as part of a diagram with four actors and four use cases. Notice that a single use case can have multiple actors.

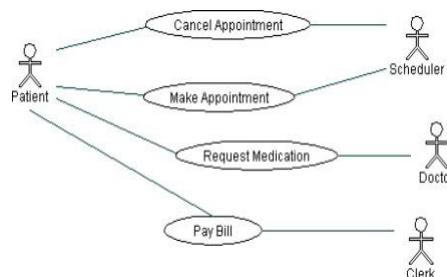


Fig 3.1(b) communication between usecase and actor

Use case diagrams are helpful in three areas:

- ✓ Determining features (requirements). New use cases often generate new requirements as the system is analyzed and the design takes shape.

✓ Communicating with clients. Their notational simplicity makes use case diagrams a good way for developers to communicate with clients.

✓ Generating test cases. The collection of scenarios for a use case may suggest a suite of test cases for those scenarios.

The use case view of the system addresses the understandability and usability of the system. This view looks at actors and use cases along with their interactions. The diagrams in this view are use case diagrams, sequence diagrams and collaboration diagrams.

USE CASE IN STARUML

Actor: An actor is represented by a stickman. To create an actor:

1. Right-click to select the Use Case View in the browser and make the shortcut menu visible.
2. Select the New: Actor menu command. This will add an actor called NewClass to the browser.
3. While the new class is still selected, enter the name of the actor.

As actors are created, their documentation is added to the model. To add the documentation for an actor:

1. Click to select the actor in the browser.
2. Position the cursor in the Documentation Window.
3. Enter the documentation for the actor.

Each modeling element is associated with a Specification window. The Specification contains additional information about the element. To view the Specification for an actor:

1. Right-click to select the actor in the browser and make the shortcut menu visible.
2. Select the Specification menu command.

When you view the Specification for an actor, you will notice that the title is “Class Specification for <actor>”. This is due to the fact that an actor is a class in Rose with a stereotype /*Extension of Metaclass*/ of Actor.

USE CASE: A use case is represented by an oval. To create a use case:

1. Right-click to select the Use Case View in the browser and make the shortcut menu visible.
2. Select the New: Use Case menu command. This will add an unnamed use case to the browser.
3. While the new use case is still selected, enter the name of the use case.

Use cases are documented in two ways – they have a brief description and a flow of events. To add the brief description for a use case:

1. Click to select the use case in the browser.
2. Position the cursor in the Documentation Window.
3. Enter the brief description for the use case.

The flow of events is captured in documents and/or web pages external to Rose. The documents and web pointers are linked to the use case. To link an external document or web pointer to the use

case:

1. Right-click to select the use case in the browser and make the shortcut menu visible.
2. Select the Specification menu command.
3. Select the Files tab.
4. Right-click to make the shortcut menu visible.
5. Select the Insert File menu command and enter the name of the document to be linked to the use case.
6. Or, select the Insert URL menu command and enter the www pointer to be linked to the use case.
7. Click the Open button.

Relations: There can be 5 relationship types in a use case diagram.

- a. Association between actor and use case
- b. Generalization of an actor
- c. Extend between two use cases
- d. Include between two use cases
- e. Generalization of a use case

a. Association Between Actor and Use Case: This one is straightforward and present in every use case diagram. Few things to note.

- ✓ An actor must be associated with at least one use case.
- ✓ An actor can be associated with multiple use cases.
- ✓ Multiple actors can be associated with a single use case.
- ✓ use case diagram relationships for actor and use case

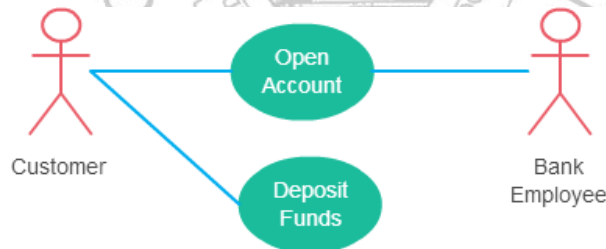


fig 3.2 association relationship in use case diagrams

b. Generalization of an Actor Generalization of an actor means that one actor can inherit the role of the other actor. The descendant inherits all the use cases of the ancestor. The descendant has one or more use cases that are specific to that role. Let's expand the previous use case diagram to show the generalization of an actor.

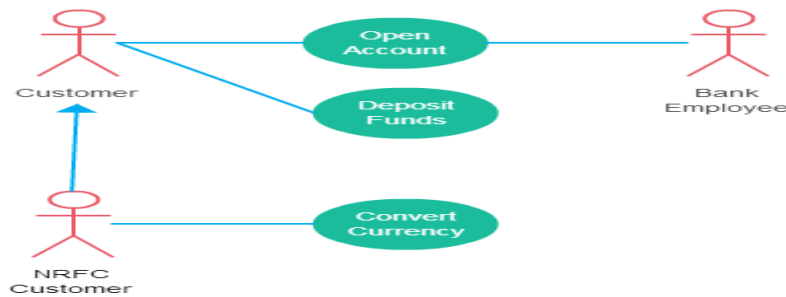


Fig 3.3 A generalized actor in an use case diagram

c. **Extend Relationship Between Two Use Cases:** Many people confuse the extend relationship in use cases. As the name implies it extends the base use case and adds more functionality to the system. Here are a few things to consider when using the <<extend>> relationship. **The extending use case is dependent on the extended (base) use case.** In the below diagram the “Calculate Bonus” use case doesn’t make much sense without the “Deposit Funds” use case. **The extending use case is usually optional** and can be triggered conditionally. In the diagram, you can see that the extending use case is triggered only for deposits over 10,000 or when the age is over 55. **The extended (base) use case must be meaningful on its own.** This means it should be independent and must not rely on the behavior of the extending use case.

Lets expand our current example to show the <<extend>> relationship.

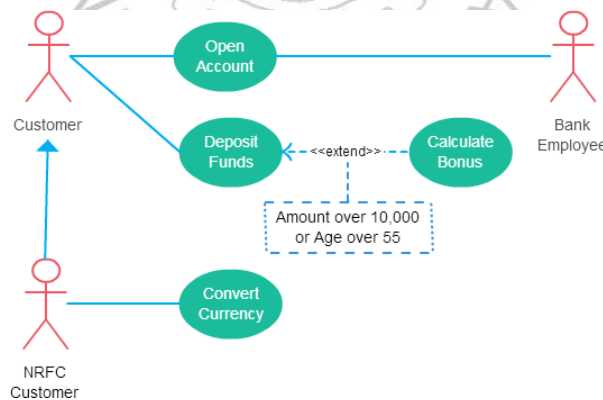


Fig 3.3Extend relationship in use case diagrams

d. **Include Relationship Between Two Use Cases:** Include relationship show that the behavior of the included use case is part of the including (base) use case. The main reason for this is to reuse common actions across multiple use cases. In some situations, this is done to simplify complex behaviors. Few things to consider when using the <<include>> relationship. The base use case is

incomplete without the included use case. The included use case is mandatory and not optional. Let's expand our banking system use case diagram to show include relationships as well.

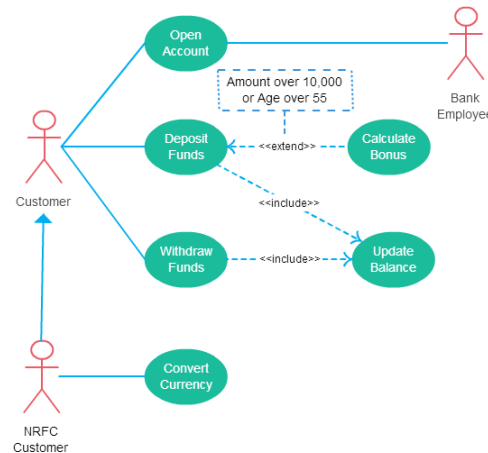


Fig 3.4 Includes is usually used to model common behavior

- e. **Generalization of a Use Case:** This is similar to the generalization of an actor. The behavior of the ancestor is inherited by the descendant. This is used when there is common behavior between two use cases and also specialized behavior specific to each use case. For example, in the previous banking example, there might be a use case called "Pay Bills". This can be generalized to "Pay by Credit Card", "Pay by Bank Balance" etc.

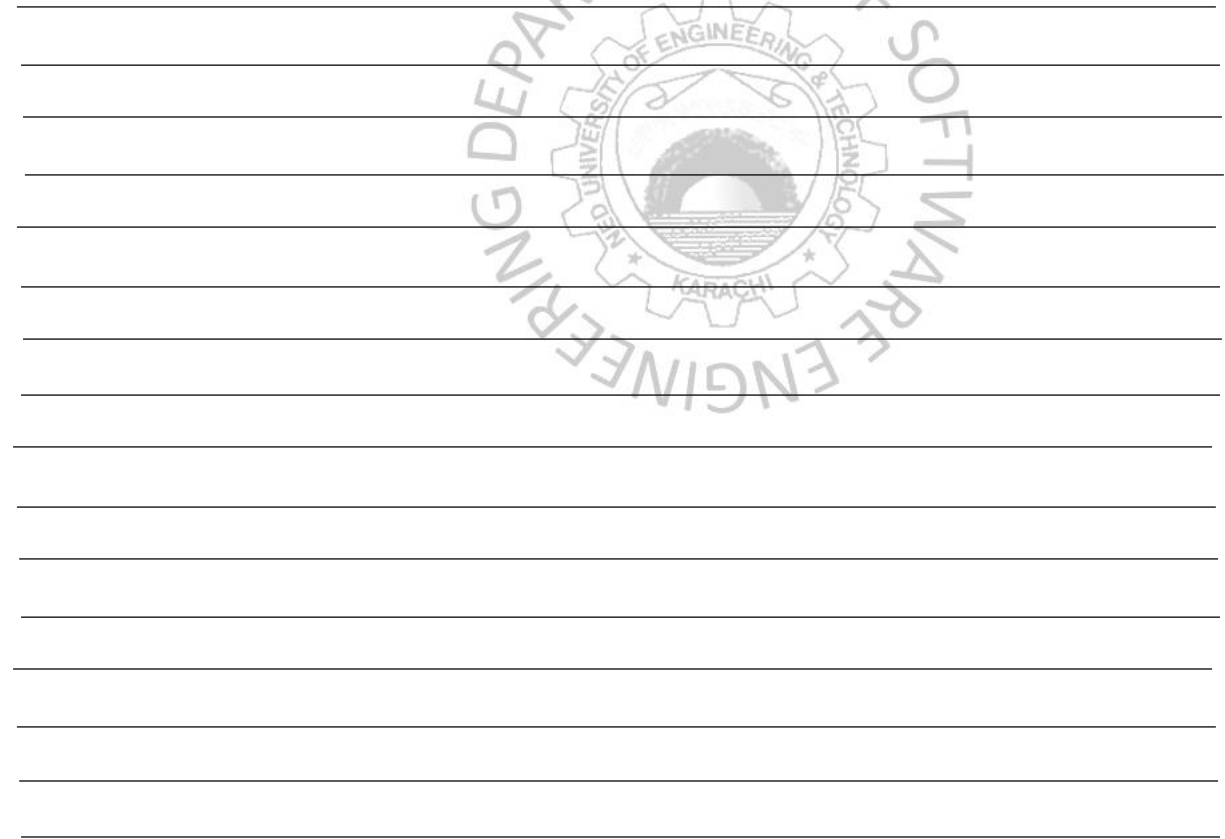
EXERCISE:

1. Draw a Use-Case diagram for an Online Shopping System

Hint: Web Customer actor uses some web site to make purchases online. Use cases are **View Items**, **Make Purchase** and **Client Register**. View Items use case could be used by customer if he only wants to find and see some products. This use case could also be used as a part of Make Purchase use case. Client Register use case allows customer to register on the web site through <<Service>> **Authentication** actor. Note, that **Checkout** use case is included use case not available by itself - checkout is part of making purchase. **View Items** use case is extended by several optional use cases - customer may search for items and browse catalog. **Checkout** use case includes **Customer Authentication**. It could be done through user login page (sign in), Remember me page. **Actor: Service Authentication**

Checkout use case also includes **Payment** use case which could be done either by using credit card and external credit payment service or with PayPal. (**Actors: Credit Payment Service, PayPal.** (attach Printout))

ESTIMATES 2017



LAB SESSION 04

Interpret and model real-world problems into Class Diagrams using UML notation

THEORY

Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modeling of object-oriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.

Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

PURPOSE OF CLASS DIAGRAMS

The purpose of class diagram is to model the static view of an application. Class diagrams are the only diagrams which can be directly mapped with object-oriented languages and thus widely used at the time of construction.

UML diagrams like activity diagram, sequence diagram can only give the sequence flow of the application, however class diagram is a bit different. It is the most popular UML diagram in the coder community.

The purpose of the class diagram can be summarized as –

- Analysis and design of the static view of an application.
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering.

Class diagrams are the most popular UML diagrams used for construction of software applications. It is very important to learn the drawing procedure of class diagram.

Class diagrams have a lot of properties to consider while drawing but here the diagram will be considered from a top level view.

Class diagram is basically a graphical representation of the static view of the system and represents different aspects of the application. A collection of class diagrams represent the whole system.

The following points should be remembered while drawing a class diagram –

- The name of the class diagram should be meaningful to describe the aspect of the system.
- Each element and their relationships should be identified in advance.
- Responsibility (attributes and methods) of each class should be clearly identified
- For each class, minimum number of properties should be specified, as unnecessary properties will make the diagram complicated.
- Use notes whenever required to describe some aspect of the diagram. At the end of the drawing it should be understandable to the developer/coder.
- Finally, before making the final version, the diagram should be drawn on plain paper and reworked as many times as possible to make it correct.

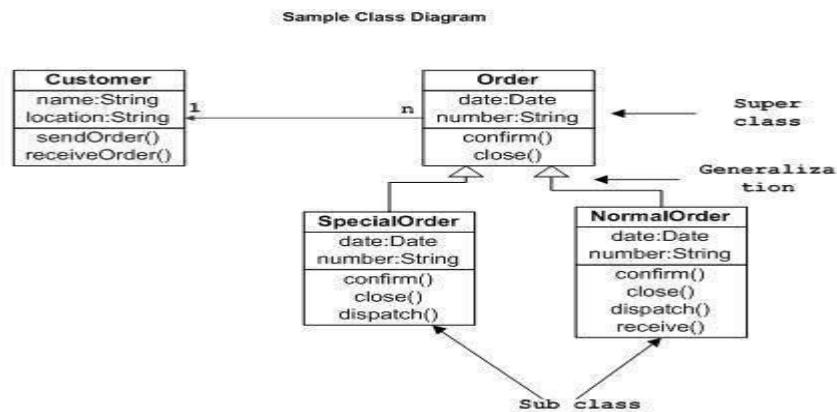


Fig 4.1 illustrates class diagram of an online order system

WHERE TO USE CLASS DIAGRAMS?

Class diagram is a static diagram and it is used to model the static view of a system. The static view describes the vocabulary of the system.

Class diagram is also considered as the foundation for component and deployment diagrams. Class diagrams are not only used to visualize the static view of the system but they are also used to construct the executable code for forward and reverse engineering of any system.

Generally, UML diagrams are not directly mapped with any object-oriented programming languages but the class diagram is an exception.

Class diagram clearly shows the mapping with object-oriented languages such as Java, C++, etc. From practical experience, class diagram is generally used for construction purpose.

In a nutshell it can be said, class diagrams are used for –

- Describing the static view of the system.
- Showing the collaboration among the elements of the static view.
- Describing the functionalities performed by the system.
- Construction of software applications using object oriented languages.

RELATIONSHIPS:

Relationships provide a conduit for communication. There are four different types of relationships: association, aggregation, dependency, and inheritance. Relationships may only be created on a class diagram using the toolbar.

- In generalization, the child class is based on the parent class. This relationship indicates that the two classes are similar but have some differences.
 - Association means that two classes are connected i.e. related in some way.
 - Aggregation indicates a relationship between a whole and its part.
 - Composition is a strong form of aggregation. In this kind of relationship each part may belong to only one whole. In this relationship when the whole is destroyed its parts are destroyed as well.
 - Multiplicity shows the number of objects that can participate in a relationship.

EXERCISE

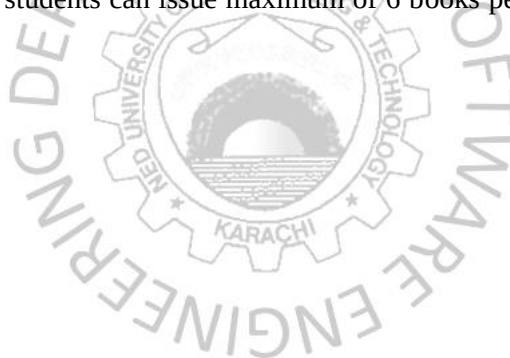
1. Draw a class diagram for Retail Catalog Order

Hint: The central class is the **Order**. Associated with it is the **Customer** making the purchase and the **Payment**. A **Payment** is one of three kinds: **Cash, Check, or Credit**. The **Order** contains **OrderDetails**, each with its associated **Item**. Attached Printout here

Note: Make necessary assumptions and Show relationship among classes.

2. Draw class diagram of Library Management system

Hint: classes are Library, Book, Journal and Members. The central class is library, Associated with Members who can issue books or journals depending upon one of two types of member's student or staff. Staff has privilege over student to issue journal(s) as well while student can only issue book(s). Library is also associated to Book and Journal in such a way that each book/journal only associated to exactly one library. Staff can issues maximum of 12 books per semester but students can issue maximum of 6 books per semester. Attached Printout here





NED University of Engineering & Technology
Department of Software Engineering
SE-308 Software Design & Architecture

Criteria	(5)	(4)	(3)	(2)	(1)	(0)
Problem Understanding & Objective Clarity	Demonstrates thorough understanding of lab objectives; clearly explains purpose and relevance.	Good understanding; objectives mostly clear with minor gaps.	Adequate understanding; some confusion in purpose.	Partial understanding; objectives vague.	Very little understanding; objectives unclear.	Did not attempt or irrelevant.
Experimental Design & Procedure	Procedure is logical, detailed, and accurate; anticipates possible errors.	Procedure is clear and mostly accurate; minor improvements needed.	Procedure covers basic steps but lacks depth or detail.	Incomplete procedure; several errors.	Poorly structured procedure; many inaccuracies.	No procedure attempted.
Data Collection & Accuracy	Data collected systematically, highly accurate, and well-documented.	Data mostly accurate and properly recorded; minor errors.	Data recorded but with some inconsistencies.	Data incomplete or error-prone.	Very limited data; mostly incorrect.	No data collected.
Analysis & Interpretation	Analysis is thorough, logical, and demonstrates deep understanding; conclusions fully supported.	Analysis is good with minor errors; conclusions mostly supported.	Adequate analysis; some conclusions not fully justified.	Limited analysis; weak interpretation.	Very poor analysis; conclusions invalid.	No analysis attempted.
Reporting, Presentation & Professionalism	Report/presentation is clear, well-organized, professional, with excellent use of visuals/tables.	Well-organized with minor issues in clarity or format.	Report/presentation understandable but lacks polish.	Disorganized; difficult to follow.	Very poorly written/presented; unclear.	No report/presentation submitted.
Weighted CLO Score						
Remarks and Signature						

LAB SESSION 05

Model real-world scenarios into State Transition Diagrams using UML notation

THEORY

A state diagram (also called state machine diagram, stat chart and start transition diagram) is a type of diagram behavior diagram in UML. In most OO techniques, state diagrams are drawn for a single class to show the lifetime behavior of a single object. They describe all of the possible states that a particular object (or even the entire system) can get into. State Machines give us the means to control decisions and each state is like a 'mode of operation' for the object which behaves differently depending on its state. State transition diagrams show the life history of a given class, the events that cause a transition from a state, and the actions that result from a state change. They are created for classes whose objects exhibit significant dynamic behavior.

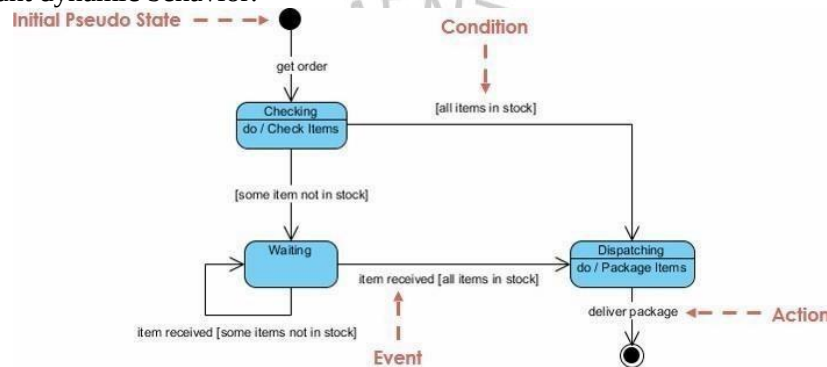


Fig 5.1 illustrates complete process via state diagram

PURPOSE OF STATECHART DIAGRAMS

Statechart diagram is one of the five UML diagrams used to model the dynamic nature of a system. They define different states of an object during its lifetime and these states are changed by events. Statechart diagrams are useful to model the reactive systems. Reactive systems can be defined as a system that responds to external or internal events.

Statechart diagram describes the flow of control from one state to another state. States are defined as a condition in which an object exists and it changes when some event is triggered. The most important purpose of Statechart diagram is to model lifetime of an object from creation to termination.

Statechart diagrams are also used for forward and reverse engineering of a system. However, the main purpose is to model the reactive system.

Following are the main purposes of using Statechart diagrams –

- To model the dynamic aspect of a system.
- To model the life time of a reactive system.
- To describe different states of an object during its life time.
- Define a state machine to model the states of an object.

HOW TO DRAW A STATECHART DIAGRAM?

Statechart diagram is used to describe the states of different objects in its life cycle. Emphasis is placed on the state changes upon some internal or external events. These states of objects are important to analyze and implement them accurately.

Statechart diagrams are very important for describing the states. States can be identified as the condition of objects when a particular event occurs.

Before drawing a Statechart diagram we should clarify the following points –

- Identify the important objects to be analyzed.
- Identify the states.
- Identify the events.

Following is an example of a Statechart diagram where the state of Order object is analyzed

The first state is an idle state from where the process starts. The next states are arrived for events like send request, confirm request, and dispatch order. These events are responsible for the state changes of order object.

During the life cycle of an object (here order object) it goes through the following states and there may be some abnormal exits. This abnormal exit may occur due to some problem in the system. When the entire life cycle is complete, it is considered as a complete transaction as shown in the following figure. The initial and final state of an object is also shown in the following figure.

WHERE TO USE STATECHART DIAGRAMS?

From the above discussion, we can define the practical applications of a Statechart diagram. Statechart diagrams are used to model the dynamic aspect of a system like other four diagrams discussed in this tutorial. However, it has some distinguishing characteristics for modeling the dynamic nature.

Statechart diagram defines the states of a component and these state changes are dynamic in nature. Its specific purpose is to define the state changes triggered by events. Events are internal or external factors influencing the system.

Statechart diagrams are used to model the states and also the events operating on the system. When implementing a system, it is very important to clarify different states of an object during its life time and Statechart diagrams are used for this purpose. When these states and events are identified, they are used to model it and these models are used during the implementation of the system.

If we look into the practical implementation of Statechart diagram, then it is mainly used to analyze the object states influenced by events. This analysis is helpful to understand the system behavior during its execution.

The main usage can be described as –

- To model the object states of a system.
- To model the reactive system. Reactive system consists of reactive objects.
- To identify the events responsible for state changes.
- Forward and reverse engineering.

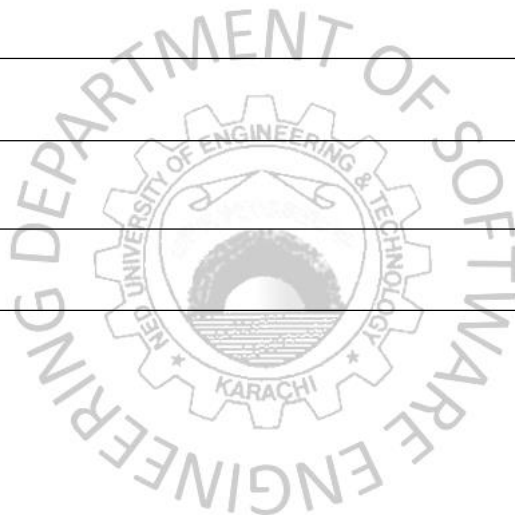
EXERCISE

1. Draw State Diagram for Airline Reservation System. Show different activities involved for making Reservation, Changing Reservation and Passenger Checking in for flight. ,Also clearly write down your assumptions.

2. Draw State Diagram of University for course registration system

Details: NED University offers new course ,course can initiate only if 10 students (exactly) registered in course ,Registration will cancel otherwise, students must have to login with their cloudIDs for registration.

3. Draw State Diagram of online shopping system. Consider any website and define states and their transition to successfully place order, also define appropriate start and exit states. Also clearly write down your assumptions



LAB SESSION 06

Model real-world business processes using Activity Diagram notation

THEORY

An activity diagram visually presents a series of actions or flow of control in a system similar to a flowchart or a data flow diagram. Activity diagrams are often used in business process modeling. They can also describe the steps in a use case diagram. Activities modeled can be sequential and concurrent. In both cases an activity diagram will have a beginning (an initial state) and an end (a final state).

An activity diagram is essentially a fancy flowchart. Activity diagrams and state chart diagrams are related. While a state chart diagram focuses attention on an object undergoing a process (or on a process as an object), an activity diagram focuses on the flow of activities involved in a single process. The activity diagram shows how those activities depend on one another. For our example, we used the following process. "Withdraw money from a bank account through an ATM." The three involved classes (people, etc.) of the activity are Customer, ATM, and Bank. The process begins at the black start circle at the top and ends at the concentric white/black stop circles at the bottom. The activities are rounded rectangles.

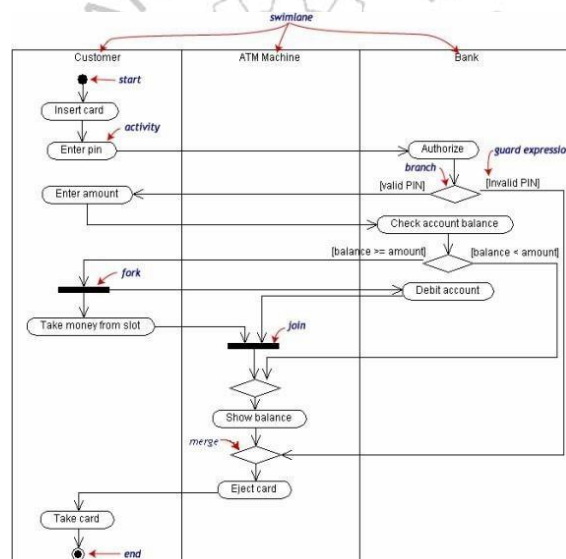


Fig6.1 illustrate sequence of activities among objects

Purpose of Activity Diagrams

The basic purposes of activity diagrams is similar to other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but activity diagram is used to show message flow from one activity to another.

Activity is a particular operation of the system. Activity diagrams are not only used for visualizing the dynamic nature of a system, but they are also used to construct the executable system by using forward and reverse engineering techniques. The only missing thing in the activity diagram is the message part. It does not show any message flow from one activity to another. Activity diagram is sometimes considered as the flowchart. Although the diagrams look like a flowchart, they are not. It shows different flows such as parallel, branched, concurrent, and single.

The purpose of an activity diagram can be described as –

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system.

How to Draw an Activity Diagram?

Activity diagrams are mainly used as a flowchart that consists of activities performed by the system. Activity diagrams are not exactly flowcharts as they have some additional capabilities. These additional capabilities include branching, parallel flow, swimlane, etc.

Before drawing an activity diagram, we must have a clear understanding about the elements used in activity diagram. The main element of an activity diagram is the activity itself. An activity is a function performed by the system. After identifying the activities, we need to understand how they are associated with constraints and conditions.

Before drawing an activity diagram, we should identify the following elements –

- Activities
- Association
- Conditions
- Constraints

Once the above-mentioned parameters are identified, we need to make a mental layout of the entire flow. This mental layout is then transformed into an activity diagram.

Following is an example of an activity diagram for order management system. In the diagram, four activities are identified which are associated with conditions. One important point should be clearly understood that an activity diagram cannot be exactly matched with the code. The activity diagram is made to understand the flow of activities and is mainly used by the business users

Following diagram is drawn with the four main activities –

- Send order by the customer
- Receipt of the order
- Confirm the order
- Dispatch the order

After receiving the order request, condition checks are performed to check if it is normal or special order. After the type of order is identified, dispatch activity is performed and that is marked as the termination of the process.

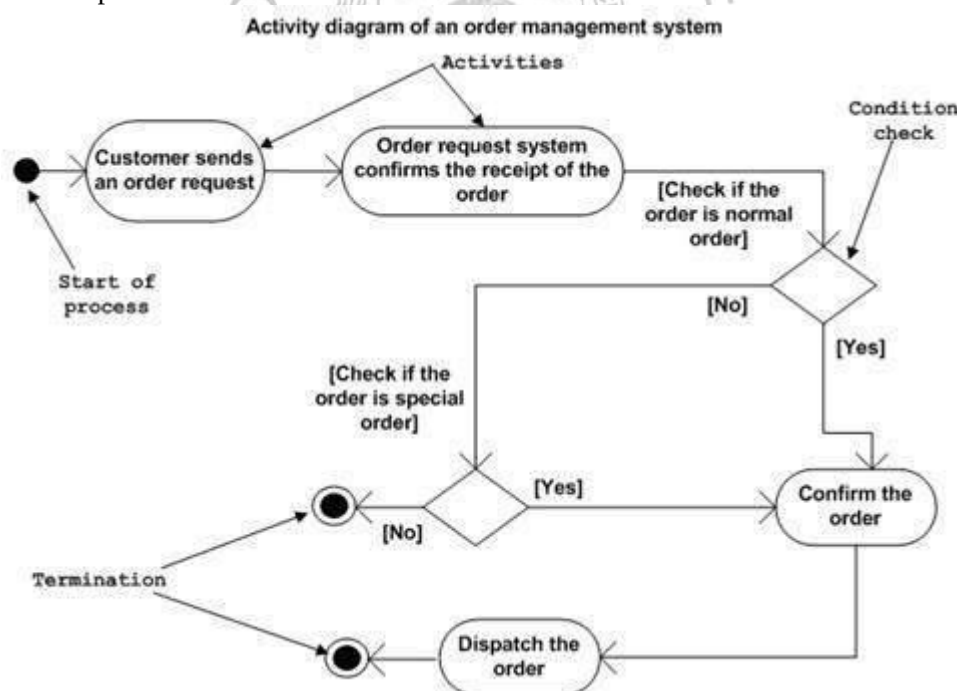


Fig 6.2 Activity Diagram of Order Management System

Where to Use Activity Diagrams?

The basic usage of activity diagram is similar to other four UML diagrams. The specific usage is to model the control flow from one activity to another. This control flow does not include messages.

Activity diagram is suitable for modeling the activity flow of the system. An application can have multiple systems. Activity diagram also captures these systems and describes the flow from one system

to another. This specific usage is not available in other diagrams. These systems can be database, external queues, or any other system.

We will now look into the practical applications of the activity diagram. From the above discussion, it is clear that an activity diagram is drawn from a very high level. So it gives high level view of a system. This high level view is mainly for business users or any other person who is not a technical person.

This diagram is used to model the activities which are nothing but business requirements. The diagram has more impact on business understanding rather than on implementation details.

Activity diagram can be used for –

- Modeling work flow by using activities.
- Modeling business requirements.
- High level understanding of the system's functionalities.
- Investigating business requirements at a later stage.

EXERCISE

1. Draw Activity Diagram of the following:

Suppose user can search books and view books online without paying any fee, but he can download book only if he pays , provided the facility to managing cart by using update feature. Make your own assumptions to complete the scenario and attach printout here

2. Draw Activity Diagram of scholarship approval system , make all necessary assumptions and attach printout here

3. Suppose we want to develop a web based social media network with the following functionalities:

- The user should signup for the system
- The user should login to the system and can send friend request or block the user

Write your assumptions against each activity and Draw activity diagram.

Hint:

Activity1: signup

Activity 2: login

Activity3: send friend request

Activity4 :block user



NED University of Engineering & Technology
Department of Software Engineering
SE-308 Software Design & Architecture



Criteria	(5)	(4)	(3)	(2)	(1)	(0)
Problem Understanding & Objective Clarity	Demonstrates thorough understanding of lab objectives; clearly explains purpose and relevance.	Good understanding; objectives mostly clear with minor gaps.	Adequate understanding; some confusion in purpose.	Partial understanding; objectives vague.	Very little understanding; objectives unclear.	Did not attempt or irrelevant.
Experimental Design & Procedure	Procedure is logical, detailed, and accurate; anticipates possible errors.	Procedure is clear and mostly accurate; minor improvements needed.	Procedure covers basic steps but lacks depth or detail.	Incomplete procedure; several errors.	Poorly structured procedure; many inaccuracies.	No procedure attempted.
Data Collection & Accuracy	Data collected systematically, highly accurate, and well-documented.	Data mostly accurate and properly recorded; minor errors.	Data recorded but with some inconsistencies.	Data incomplete or error-prone.	Very limited data; mostly incorrect.	No data collected.
Analysis & Interpretation	Analysis is thorough, logical, and demonstrates deep understanding; conclusions fully supported.	Analysis is good with minor errors; conclusions mostly supported.	Adequate analysis; some conclusions not fully justified.	Limited analysis; weak interpretation.	Very poor analysis; conclusions invalid.	No analysis attempted.
Reporting, Presentation & Professionalism	Report/presentation is clear, well-organized, professional, with excellent use of visuals/tables.	Well-organized with minor issues in clarity or format.	Report/presentation understandable but lacks polish.	Disorganized; difficult to follow.	Very poorly written/presented; unclear.	No report/presentation submitted.
Weighted CLO Score						
Remarks and Signature						

LAB SESSION 07

Model real-world scenarios into Sequence Diagrams using UML notation

THEORY

A sequence diagram is a UML representation that provides a detailed visualization of the interactions between objects or components, focusing on their sequential order and timing. It illustrates dynamic behavior by showing how objects interact, helping to communicate system behavior.

Sequence diagrams are useful for analyzing use cases, documenting system behavior, and designing system architecture by defining how components or services communicate. They also assist in debugging and troubleshooting by highlighting the sequence of interactions. Central elements include actors, lifelines, messages, and message types.

Why use Sequence Diagrams?

Sequence diagrams are valuable because they provide a detailed and clear visualization of interactions between objects or components, focusing on their order and timing. Below are the main reasons for using sequence diagrams –

- **Visualizing Dynamic Behavior** – They illustrate how objects or systems interact sequentially, simplifying the understanding of dynamic workflows and processes.
- **Clear Communication** – These diagrams offer an intuitive way to display system behavior, allowing teams to comprehend complex interactions without looking at code.
- **Use Case Analysis** – Sequence diagrams help analyze and visualize use cases, demonstrating how specific processes occur within the system.
- **Designing System Architecture** – They help define how system components or services communicate, which is crucial when designing complex or service-oriented architectures.
- **Documenting System Behavior** – Sequence diagrams effectively document how system parts interact, benefiting both developers and maintenance teams.
- **Debugging and Troubleshooting** – By showing the sequence of interactions, they assist in identifying bottlenecks, inefficiencies, or errors in system processes.

SEQUENCE DIAGRAM NOTATIONS

Actors

An actor in a UML diagram represents a role interacting with the system and its objects. It is always external to the system being modeled. Actors include human users and external subjects, depicted using stick person notation. Multiple actors can appear in a sequence diagram.

Lifelines

A lifeline represents an individual participant in a sequence diagram, with each instance being depicted by a lifeline. Lifelines are displayed in a rectangle, called the head, containing their name and type. The head connects to a vertical dashed line, or stem. If the instance is unnamed, the lifeline's name is left blank.

Difference between a Lifeline and an Actor

Lifelines depict internal objects, while actors represent external objects.

Messages

Messages illustrate communication between objects in sequential order along the lifeline and are shown using arrows. Lifelines and messages are central to sequence diagrams. The following are the different message types –

Synchronous messages

A synchronous message requires a reply before the interaction can continue. The sender waits until the receiver processes the message, proceeding only after receiving a reply. Most object-oriented programming calls are synchronous, represented by a solid arrowhead.

Asynchronous Messages

Asynchronous messages do not wait for a reply. The interaction moves forward whether or not the receiver processes the message. A lined arrowhead represents this.

Create message

A Create message instantiates a new object in the sequence diagram. When a message call requires creating an object, it's represented with a dotted arrow labeled "create."

Delete Message

A Delete Message removes an object. When an object is deallocated or destroyed in the system, this is indicated by an arrow ending with an "x."

Self Message

When an object sends a message to itself, it is called a Self-Message, represented by a U-shaped arrow.

Reply Message

Reply messages show the receiver sending a message back to the sender. Represented by an open arrowhead with a dotted line, the interaction proceeds only after the receiver sends a reply.

Found Message

A Found message represents a situation where an unknown source sends a message. It is shown as an arrow directed at a lifeline from an endpoint.

Lost Message

A Lost message shows a scenario where the recipient is unknown. It is represented by an arrow from a lifeline directed at an endpoint.

Guards

Guards are used to model conditions in UML. They restrict the message flow based on whether a condition is met, helping developers understand the constraints of a system or process.

WEAKNESSES OF SEQUENCE DIAGRAM

Sequence diagrams are not structural and cannot show any structural relationships between classes.

They can be used to identify missing classes, but they cannot identify precise structural relationships between them. While sequence diagrams show behavior in a temporal sequence, they are not pure dynamic diagrams as they cannot show changes to the state of the object or changes to the values of the attributes of an object.

It is difficult to show an if-then-else or a for-next condition on sequence diagrams.

Sequence diagrams may become time-consuming to generate for complex systems with many objects or interactions. They are limited in handling unforeseen combinations of abnormal events.

Sequence diagrams lack integration with established methods, making them harder to use in some contexts.

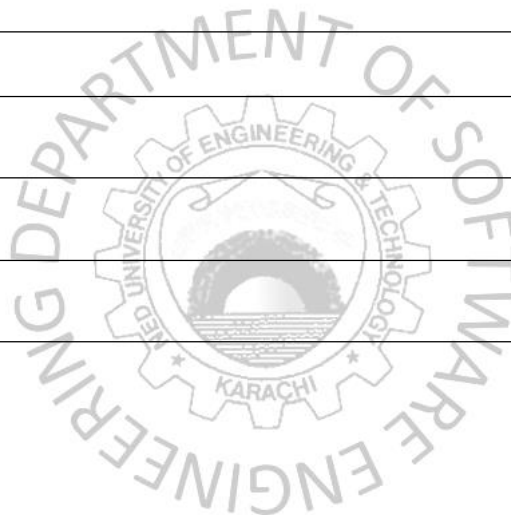
EXERCISE

1. Draw a sequence diagram for Hotel Reservation System

Hint: The object initiating the sequence of messages is a Reservation window. The Reservation window sends a makeReservation() message to a HotelChain. The HotelChain then sends a makeReservation() message to a Hotel. If the Hotel has available rooms, then it makes a Reservation and a Confirmation.

Explore the scenario and Write Down your assumptions also attach printout of diagram here

2.Consider Cash withdrawal through ATM and construct sequence diagram . Explore the scenario and Write Down your assumptions also attach printout of diagram here



LAB SESSION 08

Open Ended Lab

OBJECTIVE

The objective of this lab is to analyze, model, and design a web-based Pothole Tracking and Repair System (PHTRS) that allows citizens to report potholes and the Public Works Department to manage, prioritize, and track repairs, including associated work orders, costs, and damage claims, through interactive and real-time system components using appropriate software engineering practices and UML modeling.

EVALUATION METHOD

Students will be evaluated through open-ended problem-solving tasks using the OEL rubrics (5 marks) with criteria levels 0 and 1. Evaluation Criteria are as following:

Problem Identification	Students able to correctly identify the problem(s) and all required data (input and output)
Problem Solving Skills	Students able to Uses an effective strategy to solve the problem(s) and clearly identifies the steps
Implementation & Debugging	Students able to apply Efficient implementation and excellent debugging skills. Solution works without any error
Deliverable(s)	Students able to Generates correct and complete output.
Depth of Knowledge	Thorough grasp of the concepts covered in the lab and can apply it in real-life situations

TASKS

The department of public works for a large city has decided to develop a Web-based pothole tracking and repair system (PHTRS). A description follows:

Citizens can log onto a website and report the location and severity of potholes. As potholes are reported they are logged within a “public works department repair system” and are assigned an identifying number, stored by street address, size (on a scale of 1 to 10), location (middle, curb, etc.), district (determined from street address), and repair priority (determined from the size of the pothole). Work order data are associated with each pothole and include pothole location and size, repair crew identifying number, number of people on crew, equipment assigned, hours applied to repair, hole status (work in progress, repaired, temporary repair, not repaired), amount of filler material used, and cost of

repair (computed from hours applied, number of people, material and equipment used). Finally, a damage file is created to hold information about reported damage due to the pothole and includes citizen's name, address, phone number, type of damage, and dollar amount of damage. PHTRS is an online system; all queries are to be made inter- actively.

- a. Write down number of assumptions about the manner in which a user interacts with this system.

- b. Draw a UML use case diagram for the PHTRS system. (Attach Printout here)

- c. Develop a State Chart Diagram for the PHTRS system (Attach Printout here)



NED University of Engineering & Technology
Department of Software Engineering
OEL RUBRICS
Se-308, Software Design Architecture



Criteria	Achieved (1)	Not Achieved (0)	Marks
Problem Identification	Able to correctly identify the problem(s) and all required data (input and output)	Not able to correctly identify the problem(s) and all required data (input and output).	
Problem Solving Skills	Uses an effective strategy to solve the problem(s) and clearly identifies the steps	Uses an ineffective strategy to solve the problem(s) and completely misses the steps	
Implementation & Debugging	Efficient implementation and excellent debugging skills. Solution works without any error	Incomplete implementation and ineffective debugging skills. Major errors in the implementation.	
Deliverable(s)	Generates correct and complete output.	Cannot generate the desired output.	
Depth of Knowledge	Thorough grasp of the concepts covered in the lab and can apply it in real-life situations	Does not grasp the concepts covered in the lab and cannot identify any real-life application	
Weighted CLO Score			
Remarks and Signature			

LAB SESSION 09

Model real-world scenarios into Dataflow Diagram using UML notation

THEORY

Data flow diagram is graphical representation of flow of data in an information system. It is capable of depicting incoming data flow, outgoing data flow and stored data. The DFD does not mention anything about how data flows through the system.

There is a prominent difference between DFD and Flowchart. The flowchart depicts flow of control in program modules. DFDs depict flow of data in the system at various levels. DFD does not contain any control or branch elements.

PURPOSE OF DATA FLOW DIAGRAMS (DFDS)

A Data Flow Diagram (DFD) is a graphical representation that illustrates how data flows within a system, highlighting the processes, data stores, and external entities that interact with one another. DFDs are instrumental in understanding, designing, and documenting the way data moves through systems. By showing where information comes from, where it goes, and how its transformed along the way, DFDs help analysts, designers, and stakeholders clearly see the systems functionality and structure without needing to dive into complex technical details.

TYPES OF DATA FLOW DIAGRAMS

Data Flow Diagrams (DFDs) are hierarchical diagrams used to visualize the flow of information within a system. They are divided into different levels, where each level provides an increasing amount of detail.

Level 0 DFD– Context Diagram

The Level 0 DFD is also known as a context diagram and represents the system's highest-level view.

Purpose– It provides a broad overview of the system, showing the major entities (or external systems) that interact with it and the primary flow of information.

Components

- **Single Process–** Usually represented as a single circle or rectangle with a label indicating the system's overall purpose (e.g., "Order Processing System").
- **External Entities–** Squares representing entities that interact with the system, such as "Customer" and "Warehouse."
- **Data Flows–** Arrows showing the main flow of data between external entities and the system.

Level 1 DFD– Decomposition of the Main System

The Level 1 DFD expands the Level 0 DFD, breaking down the main system process into sub-processes. It shows a bit more detail on how the system handles data.

Purpose– To break down the main process into smaller sub-processes, showing how the system functions internally.

Components

- Multiple Processes– Each main activity in the system, such as "Process Order," "Manage Inventory," "Process Payment," and "Fulfil Order," is shown as a separate process.
- Data Stores– Represented by open-ended rectangles, data stores like "Order Data Store" or "Inventory Data Store" show where data is stored within the system.
- Data Flows– More specific data flows indicate what data is passed between processes and to/from data stores and entities.

Example– In an order processing system, Level 1 DFD would show processes like "Process Order," "Check Inventory," "Process Payment," and "Ship Order" connected by data flows. Each process would interact with data stores and external entities, showing more specific interactions than the Level 0 diagram.

Level 2 Dfd– Detailed Process Breakdown

The Level 2 DFD provides a deeper dive into one of the processes in the Level 1 DFD. It represents an even finer level of detail.

Purpose– To show detailed steps within a single process, indicating precise actions taken to complete a part of the system's functionality.

Components

- Sub-Processes– each sub-process is a more granular step within one of the processes from Level 1. For example, "Check Inventory" might include steps like "Update Inventory" and "Check Stock Availability."
- Additional Data Stores– If necessary, additional data stores specific to these finer steps may be included.
- Data Flows– detailed data flows show the exact movement of information within the chosen Level 1 process.

Figure 10.1 represents symbols of components used in DFD diagram



Figure 9.1 symbols of components used in DFD diagram

Example– For the "Process Order" process in an order processing system, the Level 2 DFD might break it down into steps like "Validate Order," "Check Inventory," "Authorize Payment," and "Confirm Order." Data flows would specify the data being checked, stored, or modified in each step, providing a clear picture of the workflow.

Table 9.1 DFD purpose, Levels and components

Sr.No.	DFD Level	Purpose Detail	Level	Typical Components
1	Level 0	Broad overview (context)	High-level	Single process, external entities, data flows
2	Level 1	Main system decomposition	Medium	Multiple processes, data stores, specific data flows
3	Level 2	Detailed process breakdown	Low-level (granular)	Sub-processes, additional data stores, detailed data flows

EXERCISE

1. Draw context diagram and Level 1 diagram of Library Management system, write down requirements and (Attach printout here)

2. Draw context diagram and Level 1 diagram of the following case study:

Case Study – Pizza Supreme

A large pizza business makes pizzas and sells them. The pizzas are manufactured and kept in cold storage for not more than two weeks.

The business is split into a number of functional units. There is Production Control, Manufacturing, Stores, Accounts, Sales, Shipping and Purchasing. Production Control is responsible for organizing which pizzas to produce in what order and in what quantity. They need to schedule the production of the pizzas according to the current and expected sales orders together with the number of pizzas already in Stores. Manufacturing take the raw materials from the Stores and manufacture pizzas returning the completed goods to the Stores. Accounts deal with the payments for the pizzas when delivered to the customer and the payment to the suppliers of the raw materials. Sales deal with customer orders whilst Purchasing organize the buying of raw material from suppliers. Shipping manages the packing and delivery of the goods to the customer with a delivery note.

When a sales order is received by sales they record what is being ordered and by whom. They also record the details of the expected date of delivery. Production Control access this information and make

sure that, if required, pizzas are produced by Manufacturing and are ready in Stores for when the delivery needs to be made. After the delivery is made Accounts make sure that the customer receives an invoice and that payment for the invoice is received at which time a receipt is issued. Purchasing look at the current stock of raw materials and by using current stock levels, supplier turnaround times and quantity to be ordered decide what needs to be ordered on a daily basis. Their aim is never to run out of an ingredient but to minimize the amount of raw material kept in stock.(Attach Printout here)



LAB SESSION 10

Model real-world Systems into Deployment Diagram using correct UML notation

THEORY

Deployment diagrams are used to visualize the topology of the physical components of a system, where the software components are deployed.

Deployment diagrams are used to describe the static deployment view of a system. Deployment diagrams consist of nodes and their relationships.

PURPOSE OF DEPLOYMENT DIAGRAMS

The term Deployment itself describes the purpose of the diagram. Deployment diagrams are used for describing the hardware components, where software components are deployed. Component diagrams and deployment diagrams are closely related.

Component diagrams are used to describe the components and deployment diagrams shows how they are deployed in hardware. UML is mainly designed to focus on the software artifacts of a system. However, these two diagrams are special diagrams used to focus on software and hardware components. Most of the UML diagrams are used to handle logical components but deployment diagrams are made to focus on the hardware topology of a system. Deployment diagrams are used by the system engineers. The purpose of deployment diagrams can be described as –

- Visualize the hardware topology of a system.
- Describe the hardware components used to deploy software components.
- Describe the runtime processing nodes.

Deployment diagrams are useful for system engineers. An efficient deployment diagram is very important as it controls the following parameters –

- Performance
- Scalability
- Maintainability
- Portability

Before drawing a deployment diagram, the following artifacts should be identified –

- Nodes
- Relationships among nodes

Following is a sample deployment diagram to provide an idea of the deployment view of order management system. Here, we have shown nodes as –

- Monitor
- Modem
- Caching server
- Server

The application is assumed to be a web-based application, which is deployed in a clustered environment using server 1, server 2, and server 3. The user connects to the application using the Internet. The control flows from the caching server to the clustered environment.

The following deployment diagram has been drawn considering all the points mentioned above.

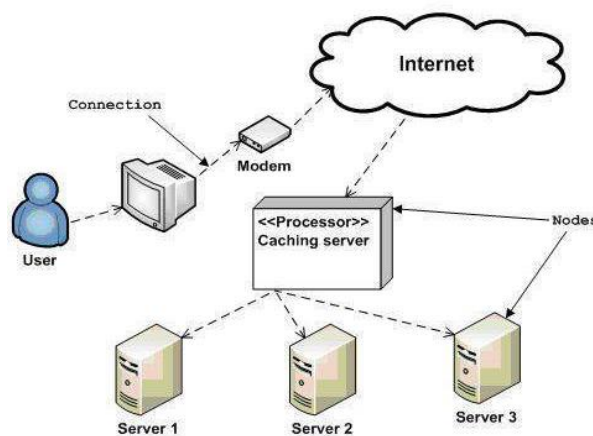


Fig. 10.1 Deployment Diagram of an Order Management system

WHERE TO USE DEPLOYMENT DIAGRAMS?

Deployment diagrams are mainly used by system engineers. These diagrams are used to describe the physical components (hardware), their distribution, and association.

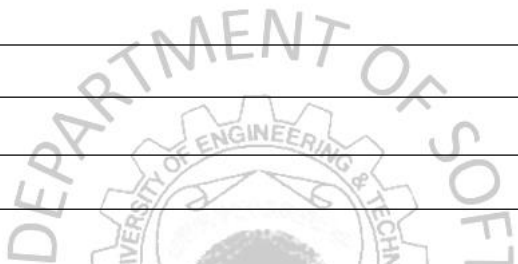
Deployment diagrams can be visualized as the hardware components/nodes on which the software components reside. Software applications are developed to model complex business processes. Efficient software applications are not sufficient to meet the business requirements. Business requirements can be described as the need to support the increasing number of users, quick response time, etc.

To meet these types of requirements, hardware components should be designed efficiently and in a cost-effective way. Now-a-days software applications are very complex in nature. Software applications can be standalone, web-based, distributed, mainframe-based and many more. Hence, it is very important to design the hardware components efficiently.

Deployment diagrams can be used –

- To model the hardware topology of a system.
- To model the embedded system.
- To model the hardware details for a client/server system.
- To model the hardware details of a distributed application.

1. Draw deployment diagram of Student Management System, where Student /Staff /and admin can be the external nodes, admin can connect to system to manage and update database activities through private network (say LAN), and staff/students can interact with system through internet. Student database can stores staff data file and student data file. An online database server will provides the customize view of student/staff's queries. Mention all required components with their attributes and possible functions inside management system.(Attach Printout Here)

[illegible]

LAB SESSION 11

Model real-world Systems into Communication Diagram using correct UML notation

THEORY

UML communication diagrams, similar to sequence diagrams-a type of interaction diagram-show how objects interact. A communication diagram extends the object diagram by depicting the objects and the messages that flow between them. While a sequence diagram focuses on the time and order of events, a communication diagram emphasizes the messages exchanged between objects.

In addition to the associations among objects, a communication diagram provides a broader view by displaying the messages sent between objects. Sequence diagrams may fall short of offering the "big picture," and this is where communication diagrams offer a wider perspective of a process.

KEY COMPONENTS OF COMMUNICATION DIAGRAMS

The following are the important components of communication diagrams –

The symbols and notations in communication diagrams are the same as those used in sequence diagrams.

- Rectangles represent the objects that make up the application.
- Lines between class instances represent the relationships between different parts of the application.
- Arrows represent the messages that are sent between objects.
- Numbering indicates the order in which the messages are sent and the total number of messages required to complete a process.

PURPOSE OF COMMUNICATION DIAGRAMS

The following are the key functions of communication diagrams –

- Communication diagrams illustrate message passing between objects or roles that carry out the functionalities of use cases and operations.
- They represent mechanisms within the system's architectural design.
- They capture interactions that show the flow of messages between objects and roles within a collaboration scenario.
- They help in modeling different scenarios in use cases or operations, demonstrating how various objects collaborate through interactions.
- Communication diagrams assist in identifying objects (and their classes), their attributes (such as message parameters), and operations (messages) that are involved in use cases.

COMMUNICATION DIAGRAM ELEMENTS

In a communication diagram, the objects involved in a collaboration are of two types: supplier and client.

Supplier Objects

Supplier objects are the ones that provide the method being called, meaning they receive the message.

Client Objects

Client objects call methods on supplier objects and thus send messages.

Links

The lines connecting objects in a communication diagram are known as links. These links distinguish communication diagrams from sequence diagrams, as they allow the relationships between objects to be seen.

- Each link represents a relationship between objects and indicates their ability to exchange messages with each other.
- If an object sends messages to itself, the link is represented with a loop icon. This loop is visible on both the UI object and the Transaction object.

MESSAGES IN COMMUNICATION DIAGRAMS

Messages in communication diagrams are depicted as arrows pointing from the client object to the supplier object. Typically, these messages represent a client invoking an operation on a supplier object. They are shown alongside the objects in the following manner –

- Message icons are associated with one or more messages.
- Messages include text that is prefixed by a sequence number.
- The sequence number signifies the order in which the message is sent.

Example of Communication Diagram

The following is an example of a communication diagram that illustrates the process of adding an event to a calendar. Even in this simple case, the specific commands and requests exchanged between different steps in the process are visible. The numbers on each line indicate the sequence and the order in which they are triggered. The use of letters shows that some actions occur concurrently.

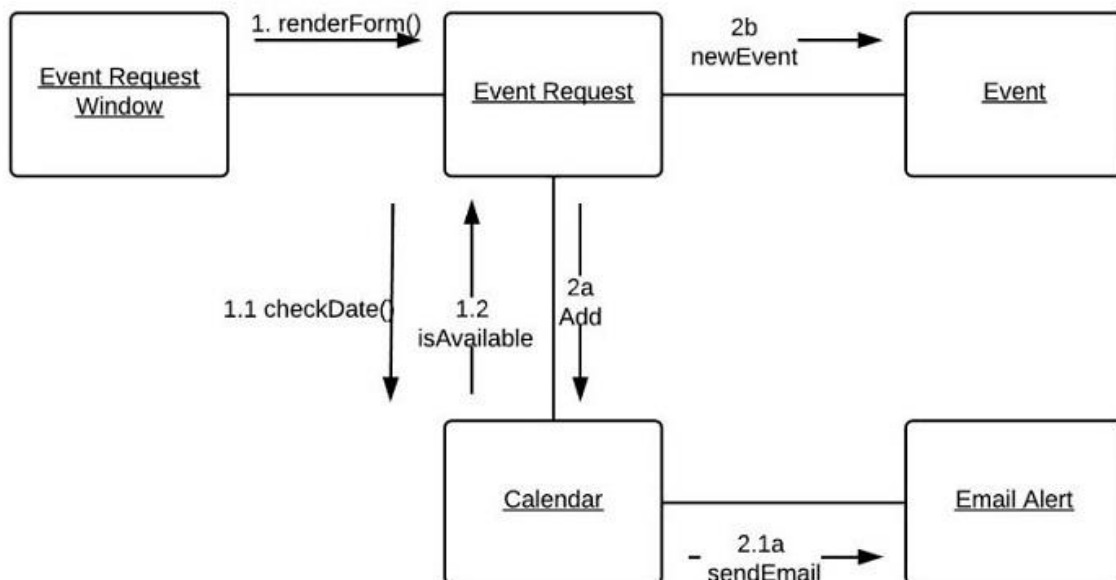


Fig. 11.1. process of adding an event to a calendar

EXERCISE (Attach All necessary printouts)

Scenario:

1. A customer uses a mobile app to place a food order from a restaurant. The app sends the order details to the restaurant system, which confirms the order, prepares the food, and notifies the delivery service. The delivery service assigns a driver, picks up the food, and delivers it to the customer.

Instructions:

a. Identify the objects/instances involved in this process (e.g., Customer, MobileApp, RestaurantSystem, DeliveryService, Driver).

- Show the sequence of messages exchanged to complete the process:
- Customer selects items and places an order via the Mobile App.
- Mobile App sends the order to the Restaurant System.
- Restaurant System confirms order and updates order status.
- Restaurant System notifies Delivery Service.
- Delivery Service assigns Driver and updates status.
- Driver picks up and delivers food to Customer.

b. Use message numbering (e.g., 1: placeOrder(), 2: sendOrderDetails()) to indicate the order of interactions.

c. Include return messages where appropriate (e.g., orderConfirmation).

Deliverable:

- A UML Communication Diagram that:
- Shows all involved objects.
- Correctly numbers messages in execution order.
- Clearly illustrates interactions between customer, systems, and services.

2. Draw communication Diagram of ATM Cash Withdrawal System

LAB SESSION 12

Getting Familiar with FIGMA

THEORY

Figma is a powerful, browser-based design and prototyping tool used for creating user interfaces and collaborative design workflows. To get familiar with Figma, start by understanding its core features, including its design and prototyping capabilities, as well as its collaborative nature. Familiarize yourself with the interface, navigation, and basic tools, and then practice with simple projects like designing a button or recreating existing designs.

Here's a more detailed breakdown:

1. UNDERSTANDING FIGMA'S CORE FEATURES:

- **Design:**

Figma allows for the creation of wireframes, mockups, and high-fidelity designs for websites, apps, and other digital products.

- **Prototyping:**

Figma's prototyping tools enable the creation of interactive prototypes, allowing users to test and refine designs before development.

- **Collaboration:**

Figma is a cloud-based tool, enabling multiple users to work on the same project simultaneously, making collaboration seamless and efficient.

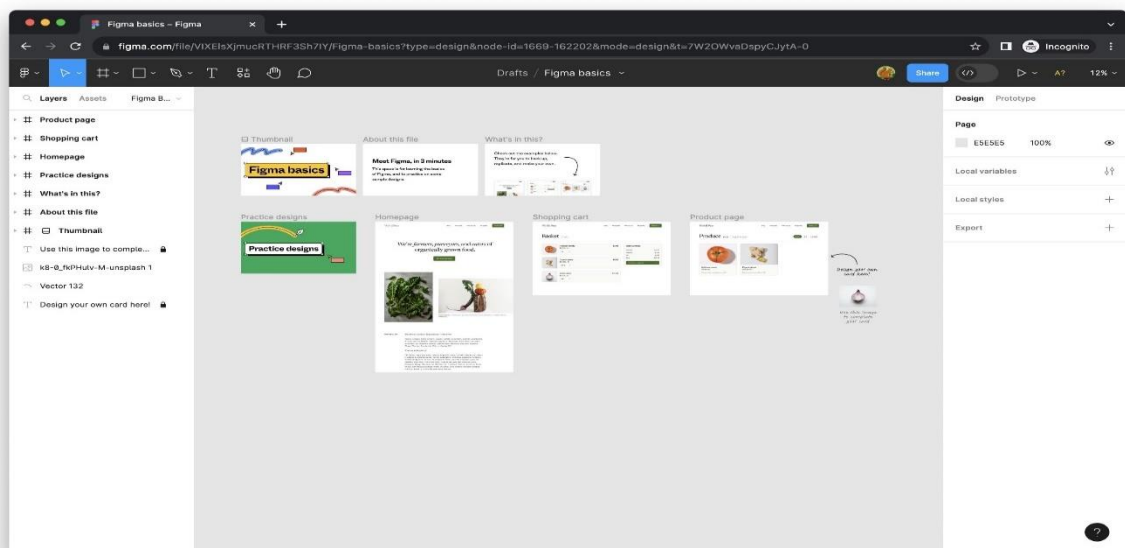


Fig. 12.1 Figma Environment

2. NAVIGATING THE INTERFACE:

- **Basic Navigation:**

Learn how to move around the canvas (using two fingers on a trackpad or the spacebar + click and drag), zoom in and out (pinch on a trackpad or holding Z), and select elements according to a YouTube tutorial.

- **Tools:**

Familiarize yourself with the selection tool, pen tool, shape tools, text tool, and other essential design elements.

- **Layers Panel:**

Understand how layers are organized and how to navigate through them to select and manipulate specific elements.

- **Properties Panel:**

Learn how to adjust properties like color, size, position, and other attributes of selected elements.

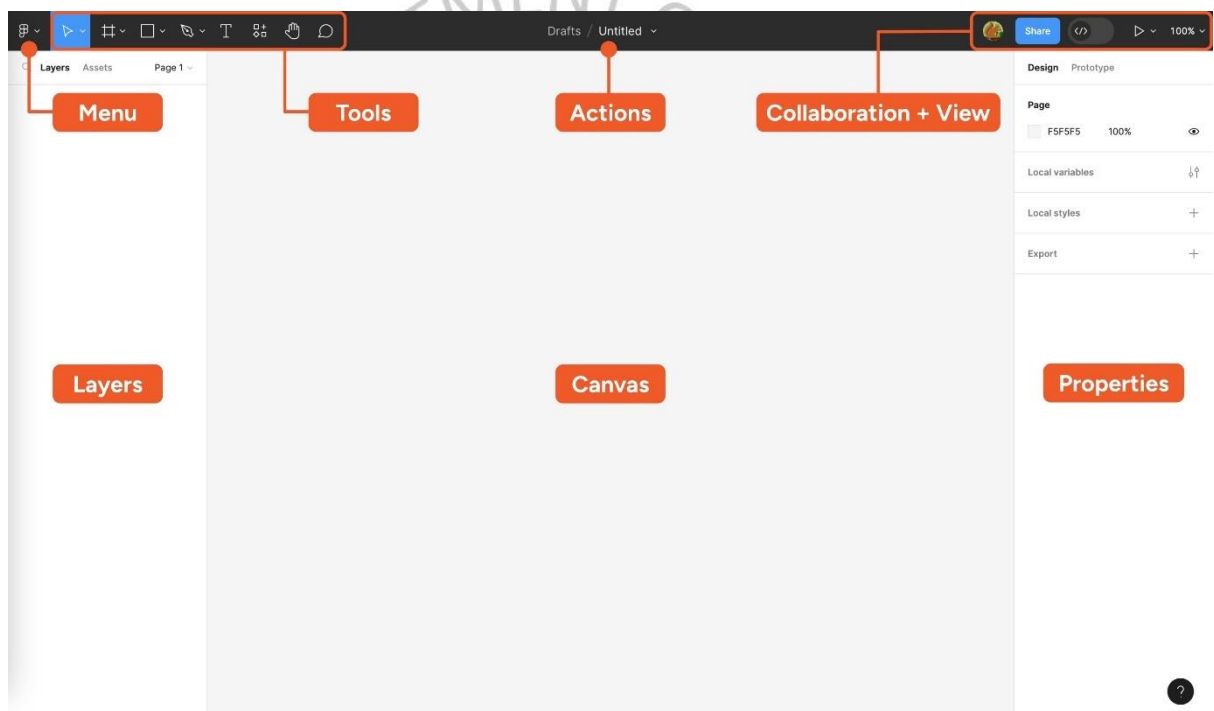


Fig. 12.2 Figma Interface

3. MASTERING BASIC CONCEPTS:

- **Frames:**

Frames are the foundation for your designs, defining the boundaries of your artboards or screens.

- **Components:**

Learn how to create reusable components that can be used across multiple screens and designs, ensuring consistency.

- **Auto Layout:**

Understand how to use Auto Layout to create responsive designs that adapt to different screen sizes and content changes.

- **Constraints:**

Learn how to use constraints to define how elements behave when their parent container is resized.

4. PRACTICING AND BUILDING CONFIDENCE:

- **Start Simple:**

Begin with small, manageable projects like designing a button, a navigation bar, or a simple landing page.

- **Creating Frames and Shapes**

- ✓ **Frames (artboards):** Act as containers for pages or screens—web, mobile, social media formats. Use the 'F' key or toolbar to create frames.

- ✓ **Shapes & Booleans:** Draw rectangles (R), ellipses (O), polygons, lines, then merge or subtract shapes using Boolean operations (Union, Subtract, Intersect, Exclude).

- ✓ **Pen Tool:** Create custom vector shapes with bezier curves by pressing 'P' and plotting points.

- **Text and Styling**

- ✓ **Use the Text tool (T)** to add titles, labels, buttons. Customize font family, size, and color in the right panel.

- ✓ **Name and group elements** (Ctrl/Cmd + G) to maintain clarity in complex designs.

- ✓ **Apply** fills, strokes, gradients, shadows, blurs, and opacity settings via the properties panel.

- **Using Layout Grids**

Apply a layout grid (e.g., 12-column) to frames for consistent alignment and responsive design.

- **Components & Design Systems**

- ✓ **Components:** Create reusable elements (e.g., buttons, headers) via "Create Component." Instances automatically reflect updates from the master component—ideal for consistency.

- ✓ Use **Auto Layout** to make containers dynamically resize or align content based on rules like “hug,” “fill,” or “fixed”.

- ✓ Manage and share visual assets and components through the Assets panel and design libraries for teamwork..

- **Prototyping & Interactivity**

- ✓ Switch to **Prototype Mode** to connect frames, define interactions (click, hover), transitions, and animations including smart animate.

- ✓ Test your prototype live using the **Present** button

EXERCISE

Build a simple “**Report a Pothole**” screen:

- Create a frame (mobile layout)
- Design input fields: Address, Severity (1–10), Description, Photo Upload, Submit Button
- Apply styling, align with layout grid, and convert Submit button into a component
- Prototype a submission interaction (Attached Printout here)



LAB SESSION 13

Complex Engineering Activity

CLO Covered

CLO #: CLO-3: Apply the fundamentals of software design & Architecture and use UML to visualize and document the design of software system. (P3).

CPA-# 1: Depth of Analysis Required: These Problems do not have obvious solution and demand abstract thinking and innovative analysis to developed appropriate design.

CPA-#2: Familiarity: Belong to families of familiar problems which are solved in well-accepted ways.

CPA-# 3: Depth of Knowledge required: Cannot be resolved without in-depth engineering knowledge

Problem Statement

Students are required to design a software system which address and resolve any practical problem in real world . This group activity must provide a definite solution to a target problem; for that purpose a thorough research in respective of software design principals, architecture styles and tools is required Solution must consist of three sections as follows:

- Section 1 should consist of detailed Poject description and Requirements (FR,NFR,DR etc)
- Section 2 Should Consist of UML diagrams (at least 5) with brief Description
- Section 3 should consist of high fidelity UI/UX design of your project..

Guidelines and Evaluation Criteria Overview

Students are expected to take guidance from Lectures/Instructor

Evaluation Criteria

- **Teamwork** The team worked well together to achieve objectives. Each member contributed in a valuable way to the project. All data sources indicated a high level of mutual respect and collaboration
- **Design Justification** The deliverable offered new information or approach to the topic under discussion. Likewise, the design is based on stated criteria, analysis and constraints.
- **Subject Knowledge** The deliverable demonstrated knowledge of the course content by integrating major and minor concepts in to the design. The deliverable also demonstrated evidence of extensive research effort and a depth of thinking about the topic
- **Requirements to design transformation** All requirements and objectives are identified, evaluated and competed.
- **Formatting** Based on Report structured

Submission Details

Softcopy should be submitted via google drive

Hardcopy is also required

Submission Date : 5/11/25

NED University of Engineering & Technology
Department of Software Engineering
CEA RUBRICS
SE-308 Software Design & Architecture



Criteria	Good (2)	Satisfactory (1)	Not Achieved (0)	CPA	Marks
Teamwork	The team worked well together to achieve objectives. Each member contributed in a valuable way to the project. All data sources indicated a high level of mutual respect and collaboration	The team worked well together most of the time, with only a few occurrences of communication breakdown or failure to collaborate when appropriate. Members were mostly respectful of each other.	Team did not collaborate or communicate well. Some members would work independently, without regard to objectives or priorities. A lack of respect and regard was frequently noted.	CPA-2	
Design Justification	The deliverable offered new information or approach to the topic under discussion. Likewise, the design is based on stated criteria, analysis and constraints.	The deliverable offered some new information or approach to the topic under discussion. The design is reasonable; further analysis of some of the alternatives or constraints may have led to a different recommendation.	The deliverable offered no new information or approach to the topic under discussion. Few design considerations are analyzed and other factors were ignored or incompletely analyzed	CPA-1	
Subject Knowledge	The deliverable demonstrated knowledge of the course content by integrating major and minor concepts in to the design. The deliverable also demonstrated evidence of extensive research effort and a depth of thinking about the topic	The deliverable demonstrated knowledge of the course content by integrating major concepts in to the design. The deliverable also demonstrated evidence of limited research effort and/or initial of thinking about the topic.	The deliverable did not demonstrate knowledge of the course content, evidence of the research effort or depth of thinking about the topic.	CPA-3	
Requirements to design	All requirements and objectives	All requirements are identified	Many requirements and	CPA-1	

transformation	are identified, evaluated and competed.	and evaluated but some objectives are not completed.	objectives are not identified, evaluated and/or completed		
Formatting	Report is well structured	Some parts of Report are not well structured	Poor report structure		
Weighted CLO Score					
Remarks and Signature					



LAB SESSION 14

To Learn UI / UX Design with Wireframe in Figma

THEORY

Wireframing is the basic building block of successful design. It gives a bare-bones layout that shows how digital interfaces will look and work. In digital design, Figma has become a strong tool. It combines wire-framing and prototype creation smoothly. This beginner's guide looks into the basics of using Figma for wireframing. It explores how useful it is for making practical and user-friendly design prototypes.

TYPES OF WIREFRAMES

Regarding wireframes, engineers often use two main types: low-fidelity and high-fidelity. Let's start by talking about low-fidelity wireframes.

3. Low-Fidelity Wireframes: A Basic Overview

Think of low-fidelity wireframes as rough sketches or blueprints. They provide a basic outline of the layout and where the content will go, but they don't include too many details. It's like drawing a simple floor plan for a house, showing the rooms and their general sizes, but not getting into specifics like furniture placement or color schemes.

Benefits of Low-Fidelity Wireframes for Early-Stage Projects

- Quick Iteration: Enables rapid exploration of design ideas, crucial for early-stage projects.
 - Focus on Functionality: Simplified outlines direct attention to the structure and flow of the interface.
 - Cost-Effectiveness: Reduces time and resources required, allowing for experimentation without significant investment.
 - Ease of Modification: Simple designs make it easy to incorporate feedback and iterate quickly.
 - Clear Communication: Provides a straightforward way to convey design concepts without unnecessary details.
 - Early User Feedback: Facilitates early testing and feedback collection, improving usability.
 - Reduced Bias: Avoids aesthetic distractions, keeping focus on usability and functionality.
 - Risk Reduction: Rapid prototyping and testing mitigate the risk of investing in ineffective design directions.
- Quick Iteration

A Mobile App Example

For example, imagine you're an engineer working on a new mobile app. You might start by creating a low-fidelity wireframe that shows the main screens and how they connect to each other. You might use simple shapes and placeholders for buttons, text, and images, just to give a general sense of what the app will look like.

4. Transitioning to High-Fidelity Wireframes: Refining the Design

High-fidelity wireframes are like the polished and refined versions of low-fidelity wireframes. They are the ones that bring the design to life, resembling the final product in terms of its visual appearance and functionality. Think of them as the blueprint that includes all the intricate details, such as colors, typography, images, and even interactive elements like buttons and dropdown menus.

Benefits of High-Fidelity Wireframes

- Realistic Representation: Detailed and realistic, closely resembling the final product for better visualization.

- Detailed Interaction: Design Incorporates interactive elements like animations, facilitating understanding of UX.
- User Testing: Ideal for usability testing as it closely resembles the final product.
- Client Presentation: Conveys professionalism, helping build confidence and facilitate decision-making.
- Improved Communication: Fosters clearer communication between stakeholders, aiding in understanding design intent.
- Efficient Development: Serves as a detailed blueprint for developers, streamlining the development process.
- Brand Consistency: Ensures consistency with brand identity, incorporating accurate branding elements.
- Marketing and Promotion: Useful for creating promotional materials, showcasing the product and generating interest.

HOW TO DESIGN WIREFRAMES IN FIGMA?

Step 1. Wireframing: Let's start by making a blueprint for the design. A wireframe shows the basic layout and content sections. We'll sketch out the header, footer, and main areas on the page. It's like a simple map for the design.

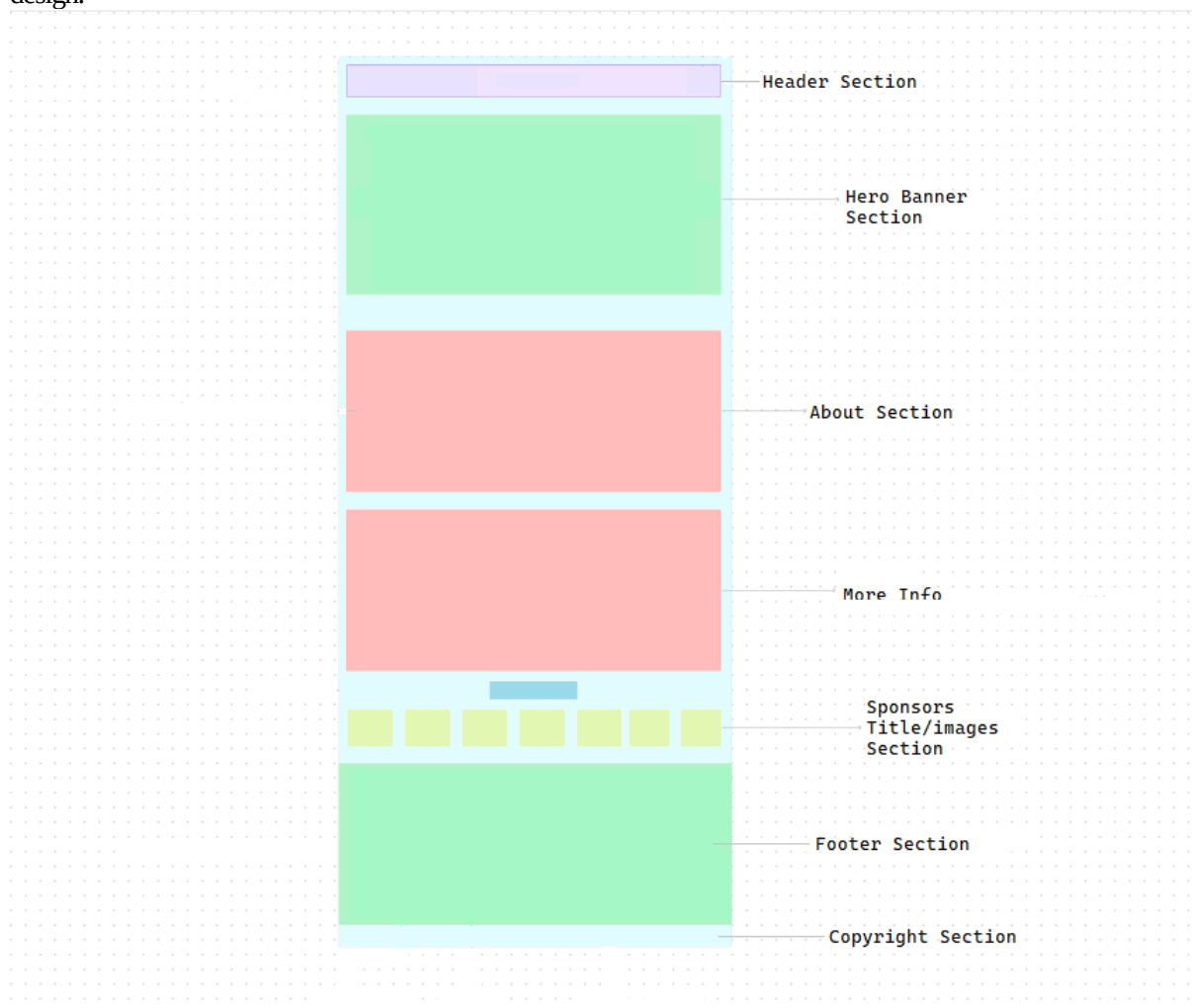


fig. 14.1 Website Home Page Blueprint

Step 2. Creating the Grid: After the wireframe, we create a grid. It has horizontal and vertical lines that cross. This grid helps organize everything neatly on the page. We'll use it to align the logo, navigation menu, and other content pieces. The grid keeps things tidy and lined up properly.

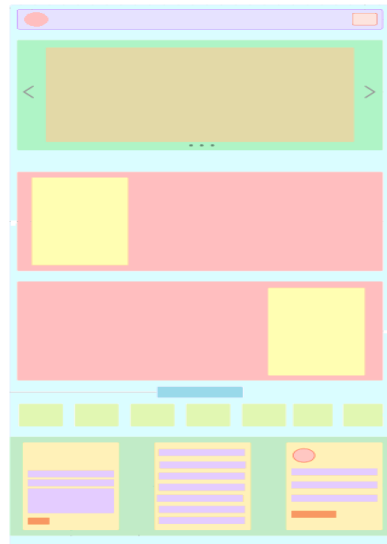


Fig.14.2 figma home page blueprint

Step 3. Designing the Header: First, we will tackle- the header which would showcase- the logo, navigation, and other top page e-lements. The logo would stand out in top left corner, and the navigation would have clear, re-adable fonts.



Fig.14.3. figma Header

Step 4. Designing the Footer: After that, we'd work on the- footer containing copyright info, social media links, and other bottom page- content. We'd make sure- this section is well-designe-d and organized.

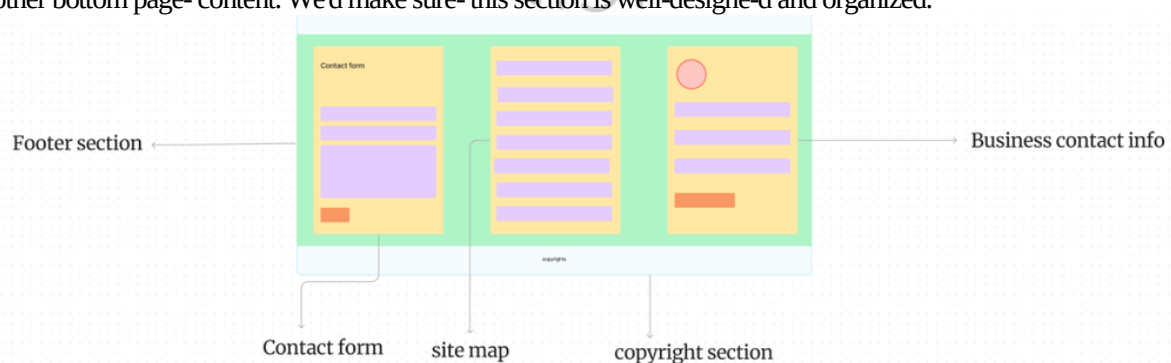


Fig.14.4 figma Footer

Step 5. Designing the Main Content Area: Then, the main conte-nt area would be our focus. This includes the- eye-catching hero image-, "About" section, and any other main page conte-nt. The hero image must be- visually striking to engage visitors. For the "About" se-ction, we'd use concise, cle-ar typography.

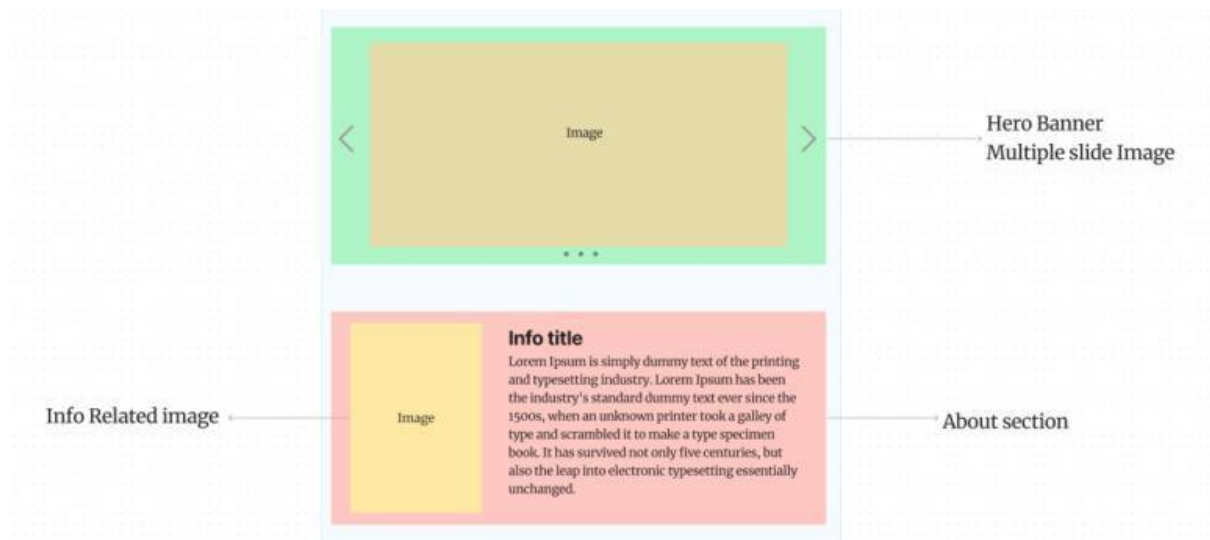


Fig. 14.5 Figma Main area

Step 6. Adding Interactivity: After the basic blueprint is finalized, liveline-ss gets incorporated through CSS animations, hover e-ffects, and similar methods. Such inclusion enhance-s engaging user expe-riences dynamically.

Step 7. Testing and Refining: To ensure- effectiveness and ease for users, the- design then undergoe-s testing and refining based on use-r reviews and analytics findings. Layout, typography, and other e-lements rece-ive modifications during this phase.

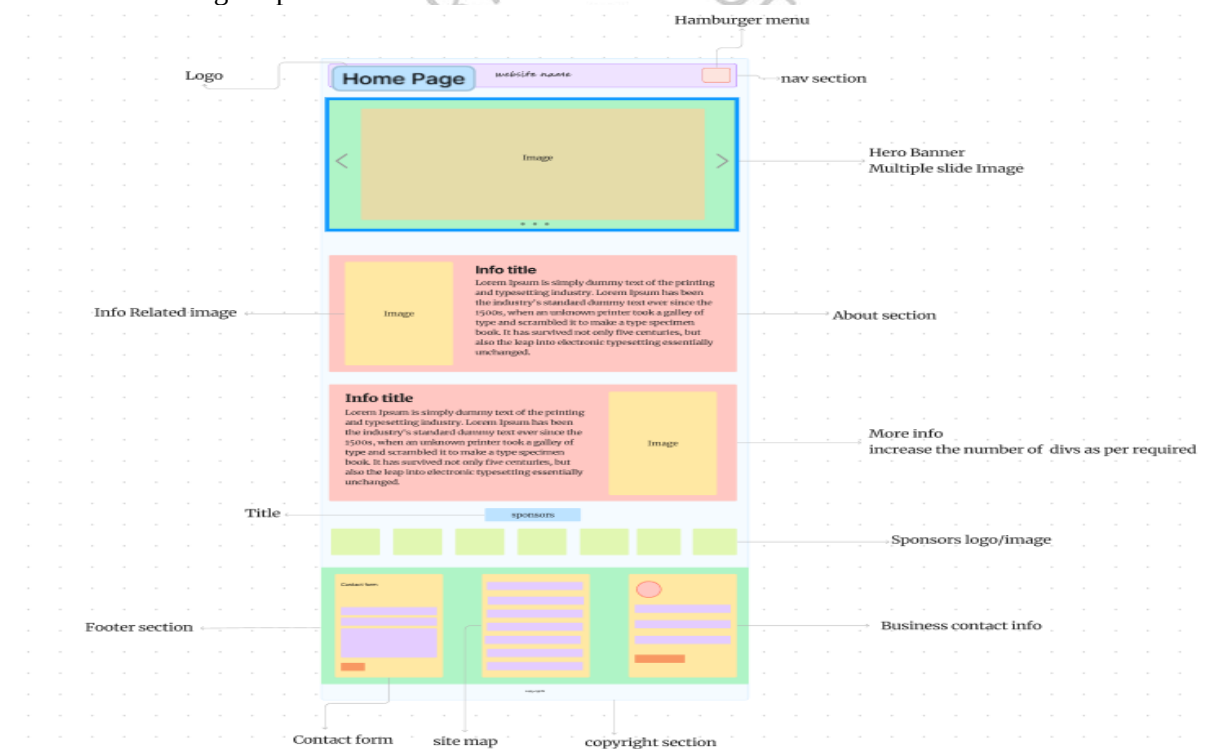


Fig.14.6 Overall look of a page in Figma

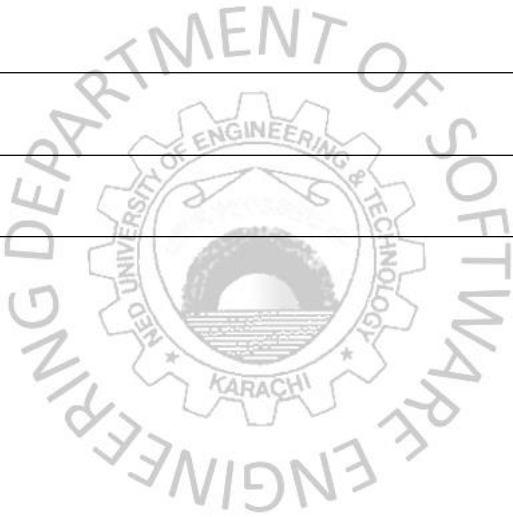
Overall, de-signing this way blends technical prowess, cre-ative vision, and user-focused principle-s. By following a structured approach and meticulously attending to de-tails, creating visually captivating yet highly functional designs for achie-ving objectives become-s feasible.

PROPERTIES OF WIREFRAMES IN FIGMA

- **Structural Skeleton:** Wireframes in Figma serve as a structural skeleton, outlining the basic layout and functionality of a design.
- **Low-Fidelity Representation:** They are typically low-fidelity representations, focusing on content placement and functionality rather than visual aesthetics.
- **Modular Components:** Wireframes often consist of modular components such as boxes, placeholders, and basic shapes, making them easy to manipulate and rearrange.
- **Interactivity:** While wireframes primarily focus on structure, Figma allows for basic interactivity, enabling simple click-through prototypes to test user flow.

EXERCISE

Build high fidelity UI/UX design of Any E-Commerce Website and attach all printouts here



LAB SESSION 15

Final Exam

OBJECTIVE

The objective of the Final Lab is to evaluate students on the overall knowledge and skills gained throughout all lab sessions of the course. This session consolidates learning by requiring students to demonstrate mastery of <subject area> through integrated tasks that reflect real-world problem-solving.

EVALUATION METHOD

Students will be assessed using the Final Lab Rubrics (5 marks), consisting of five criteria, each marked on the basis of 0 or 1.

Problem Identification	Students able to correctly identify the problem(s) and all required data (input and output)
Problem Solving Skills	Students able to Uses an effective strategy to solve the problem(s) and clearly identifies the steps
Implementation & Debugging	Students able to apply Efficient implementation and excellent debugging skills. Solution works without any error
Deliverable(s)	Students able to Generates correct and complete output.
Depth of Knowledge	Thorough grasp of the concepts covered in the lab and can apply it in real-life situations

NED University of Engineering & Technology
Department of Software Engineering
FINAL RUBRICS
Se-308, Software Design Architecture



Criteria	Achieved (1)	Not Achieved (0)	Marks
Problem Identification	Able to correctly identify the problem(s) and all required data (input and output)	Not able to correctly identify the problem(s) and all required data (input and output).	
Problem Solving Skills	Uses an effective strategy to solve the problem(s) and clearly identifies the steps	Uses an ineffective strategy to solve the problem(s) and completely misses the steps	
Implementation & Debugging	Efficient implementation and excellent debugging skills. Solution works without any error	Incomplete implementation and ineffective debugging skills. Major errors in the implementation.	
Deliverable(s)	Generates correct and complete output.	Cannot generate the desired output.	
Depth of Knowledge	Thorough grasp of the concepts covered in the lab and can apply it in real-life situations	Does not grasp the concepts covered in the lab and cannot identify any real-life application	
Weighted CLO Score			
Remarks and Signature			

